Gymnase de Nyon

Travail de Maturité 2024

Comment créer et optimiser un système à base d'un microprocesseur pour un jeu vidéo comme Tetris

ADRIAN VLASOV

Classe : 3M02

Partie principale

Pages 1-33

Direction : M. Thomas Morel

Expert : M. René Beuchat

Février 2024

Table des matières

Table des matières **1**

1 Introduction **2**

2 Résumé **3**

3 Approche et Choix **4**
3.1 Historique et choix d'approche 4
3.2 Indicateurs 5
3.3 Interruptions 6
3.4 Mémoire et bus de données 6
3.5 L'utilisation d'un FPGA 7

4 Fonctionnement **8**
4.1 Lexique 8
4.2 Notions préliminaires 8
4.3 Implémentations 8
4.4 Détails du fonctionnement du processeur 9
4.5 Parties du processeur 9
4.6 Prototype 13
4.7 Quelques détails 15

5 Programmation du compilateur d'assembleur **16**
5.1 Introduction de l'assembleur 16
5.2 Fonctionnement du compilateur 17

6 Programmation de Tetris **18**
6.1 Programmation en C 18
6.2 Retranscription de l'algorithme en assembleur 19
6.3 Organisation de la mémoire 23

7 Implémentation du son **23**
7.1 Fonctionnement physique 23
7.2 Comment marche un son synthétisé 24
7.3 Le calcul du son 24
7.4 Jeu de son 25

8 Résultats obtenus **25**
8.1 Rappel 25
8.2 Détails techniques 26
8.3 Comparaison avec d'autres processeurs 27
8.4 Analyse des résultats 28

9 Conclusion **29**
9.1 Rappel 29
9.2 Critique 29
9.3 Pour aller plus loin 30

10 Remerciements **30**

11 Bibliographie **31**

12 Annexes **34**
12.1 Détails de l'implémentation 34
12.2 Problèmes rencontrés 34
12.3 Format du jeu d'instructions 34
12.4 Champs de bits 35
12.5 Jeu d'instructions 39
12.6 Interview 43
12.7 Code Source et Fichiers 44

1 Introduction

La loi de Moore [1] affirme que le nombre de transistors dans un circuit intégré double chaque deux ans avec une augmentation non significative des coûts de production. C'est ce que Gordon Moore (Figure 1), cofondateur d'Intel, a prédit en 1975. Cette observation est même restée valable jusqu'à aujourd'hui. Mais le futur est inévitable ; nous ne pouvons point continuer dans cette direction. En effet, la taille des transistors est bornée par des contraintes physiques, comme la taille des atomes, [2] ou comme la façon dont les transistors sont faits, [3] et donc la loi de Moore ne pourra pas continuer à être valable pour toujours.



FIGURE 1 – Gordon Moore chez Intel

Nous ne pouvons donc plus compter sur la capacité des processeurs à faire tourner nos programmes non optimisés encore plus rapidement qu'avant. Nous allons donc devoir nous préparer à optimiser [4] les applications.

Du point de vue environnemental, les consommateurs, en voulant toujours le nouveau, jettent leurs anciens appareils qu'ils trouvent trop lents, alors que ceux-ci pourraient encore être utilisés si les programmes étaient plus optimisés. [5] De plus, une grande partie des multinationales soutiennent ce phénomène en abandonnant la prise en charge des appareils plus anciens. Un grand exemple de ce fait serait Microsoft, avec le système d'exploitation très populaire Windows 11, qui a abandonné la prise en charge de mises à jour d'ordinateurs d'un peu plus que quinze ans, ce qui forcerait les consommateurs à acheter un nouvel ordinateur. De plus, les applications non optimisées utilisent plus d'électricité pour tourner [6], [7] ce qui donne un impact négatif sur l'environnement en raison du gaspillage d'électricité.

D'un point de vue financier, optimiser des applications pour cent ou deux-cents appareils différents coûtera plus cher que d'augmenter la vitesse du processeur de toutes les machines. [8] Pour illustrer ce fait, intéressons-nous au cas d'Android, le système d'exploitation largement utilisé. La majorité des fabricants de téléphones utilisent Android (Samsung, par exemple [9]), et il y a une grande variété de smartphones Android sur le marché. Optimiser une application pour chaque appareil coûtera donc plus cher que d'augmenter la vitesse du processeur de toutes les machines. C'est pourquoi les développeurs ne veulent pas optimiser leurs programmes pour chaque téléphone. [10] Pour faire face au problème de la lenteur des logiciels, le fabricant de l'appareil en question équipe ses produits avec un processeur encore plus performant que la génération précédente. Par exemple, Samsung mentionne que les téléphones deviennent plus rapides [11] et que de mettre un nouveau processeur aidera à réduire les lenteurs.

Auparavant, les programmes étaient davantage optimisés et les programmeurs essayaient de bien utiliser toutes les ressources mises à disposition par le fabricant de l'appareil en question, [12] mais maintenant, les programmeurs n'ont plus besoin de se soucier de la vitesse des processeurs parce que ceux-ci sont devenus tellement rapides que la plupart des applications tournent très bien sans optimisation.

Dans ce travail de Maturité, nous allons construire un système à partir de zéro pour montrer qu'en optimisant les programmes, on peut avoir un système très rapide avec un processeur très simple. Afin de

1. INTEL CORPORATION. *Moore's Law - The past, present and future of Gordon Moore's golden rule for the semiconductor industry. [En Ligne].* Page consultée le 17 Mars 2024. URL : https://www.intel.com/content/www/us/en/newsroom/resources/moores-law.html.
2. NIDEC GROUP. *Calculating Infinity : The Paradox Of Moore's Law [En Ligne].* Page consultée le 26 Mars 2024. URL : https://acim.nidec.com/drives/control-techniques/news-and-media/blog/insights/calculating-infinity-the-paradox-of-moores-law.
3. Arnab Hazari THE CONVERSATION MEDIA GROUP LTD. *The future of electronics is light.* Page consultée le 27 Mars 2024. URL : https://theconversation.com/the-future-of-electronics-is-light-68903.
4. Dans ce contexte, le terme ≪optimiser≫ désigne le fait d'améliorer l'efficacité des applications afin d'accroître leurs performances tout en utilisant les mêmes ressources système qu'auparavant.
5. PCWORLD - IDG COMMUNICATIONS, INC. *Shopping for an older CPU ? Don't go past this—or you'll regret it.* Page consultée le 27 Mars 2024. URL : https://www.pcworld.com/article/539090/shopping-for-an-older-cpu-dont-go-past-this-or-youll-regret-it.html.
6. David CASSEL. *Which Programming Languages Use the Least Electricity ?* Page consultée le 27 Mars 2024. URL : https://thenewstack.io/which-programming-languages-use-the-least-electricity/.
7. Muhammad Umair KHAN et al. *Measuring power consumption in mobile devices for energy sustainable app development : A comparative study and challenges.* Page consultée le 27 Mars 2024. URL : https://www.sciencedirect.com/science/article/abs/pii/S2210537921000780.
8. Ayush Jalan WWW.MAKEUSEOF.COM. *Why Do So Many Developers Launch Their Apps on iOS Before Android ?* Page consultée le 27 Mars 2024. URL : https://www.makeuseof.com/why-are-cross-platform-apps-released-on-ios-first/.
9. SAMSUNG. *One UI Beta Program - Experience the latest One UI with Android 14.* Page consultée le 27 Mars 2024. URL : https://developer.samsung.com/one-ui-beta.
10. WWW.MAKEUSEOF.COM, *Why Do So Many Developers Launch Their Apps on iOS Before Android ?*, op. cit.
11. SAMSUNG. *Samsung Galaxy S24 Ultra.* Page consultée le 27 Mars 2024. URL : https://www.samsung.com/us/smartphones/galaxy-s24-ultra/.
12. Patrik TENNBERG. *The lost art of code optimization.* Page consultée le 27 Mars 2024. URL : https://www.linkedin.com/pulse/lost-art-optimization-patrik-tennberg.

valider le fonctionnement de ce processeur, nous ferons tourner le jeu Tetris [13] dessus. Puis, nous allons comparer notre implémentation à celles de quelques ordinateurs de nos jours en termes de vitesse, à l'aide de paramètres comme la vitesse de l'horloge et le nombre d'instructions traitées par coup d'horloge, mais aussi en termes de consommation d'électricité.

Pour réaliser ce travail ambitieux, il conviendra de statuer sur plusieurs problématiques. Comment organiser le processeur, c'est-à-dire choisir une architecture efficace pour cette implémentation ? Quels compromis faudrait-il faire pour avoir un processeur très simple, mais en même temps rapide ? Pour parvenir à la réponse de ces questions, nous allons élaborer une liste minimale d'instructions pour notre processeur, ce qui conduira aux différentes parties à implémenter (par exemple ALU, [14] connexions externes...). Pour tester Tetris sur notre système, nous allons connecter à notre processeur des périphériques comme un écran, quelques boutons et un haut-parleur. Enfin, à l'aide de celui-ci, nous ajouterons du son à notre jeu.

Dans une dernière partie, nous comparerons notre implémentation à celles de quelques ordinateurs utilisés de nos jours.

2 Résumé

Nous nous sommes demandés si un processeur, même avec une fréquence d'horloge aussi lente que $40MHz$ (cette fréquence est lente de nos jours, mais beaucoup de processeurs fonctionnaient à 8MHz dans les années 80), pouvait faire tourner un jeu comme Tetris, parce que nous l'avons écrit en assembleur. Ceci s'est avéré possible et donc nous avons créé un processeur complet capable de faire tourner Tetris avec du son. Pour ce faire, nous avons implémenté un processeur à 64 bits et un compilateur d'assembleur pour pouvoir programmer le jeu Tetris en assembleur. Nous avons également implémenté le son pour ce jeu.

Notre système utilise le circuit imprimé CMOD-A7 de Digilent, qui contient une FPGA Artix-7. Nous avons utilisé le logiciel Vivado pour programmer la FPGA, puisqu'elle a été fabriquée par Xilinx.

Étant implémenté sur un FPGA, notre processeur a une fréquence d'horloge de $40MHz$ et est capable de faire tourner Tetris sans problème. Il a 8 entrées et sorties GPIO, dont les 2 dernières sont des sorties PWM pour le son. Ces sorties PWM sont l'inverse l'une de l'autre. La résolution de ces sorties PWM est de 8 bits. Notre processeur a 16 registres de 64 bits.

Nous avons également comparé notre processeur avec d'autres processeurs, comme le *ESP32* ou le *STR71xF*. Le nombre d'instructions par coup d'horloge de notre processeur (MIPS/MHz) s'est avéré plus grand (1.0) que celui du *STR71xF* (0.9) et celui du *ESP32* (0.6). Nous avons également mesuré la consommation énergétique de notre processeur, qui est de $0.311W$. Nous sommes donc parvenus à créer un processeur très simple, mais complet, capable de faire tourner Tetris avec du son, avec une consommation énergétique très faible. Nous avons atteint notre objectif.

13. Joel Lee WWW.MAKEUSEOF.COM. *30 Years Of Tetris : A History Of The World's Most Beloved Puzzle Game*. Page consultée le 27 Mars 2024. URL : https://www.makeuseof.com/tag/30-years-tetris-history-worlds-beloved-puzzle-game/.

14. LENOVO. *ALU : Arithmetic Logic Unit Key Components & Logic Gates*. Page consultée le 7 Avril 2024. URL : https://www.lenovo.com/ph/en/glossary/arithmetic-logic-unit/.

3 Approche et Choix

Pour concevoir un processeur, il faut tout d'abord connaître le but. Certains font des processeurs permettant de rapidement calculer des fonctions comme sinus et cosinus pour des cartes graphiques (comme Nvidia par exemple), d'autres inventent une architecture pour superordinateurs (comme PowerPC). Notre but ici est de créer le processeur le plus simple qui devrait faire tourner Tetris, puis de comparer sa vitesse aux processeurs grand public d'aujourd'hui (comme ATmega328P [15] de l'Arduino [16] par exemple).

3.1 Historique et choix d'approche

Notons que, historiquement, les processeurs étaient d'abord dessinés à la main sur du Mylar (type de feuille fine en polyester), puis transférés sur du «Rubylith» (Figure 2). Le Rubylith est un matériau à deux couches, dont la couche de base est du Mylar transparent qui est recouvert d'une couche fine d'un matériau ressemblant à du cellophane rouge. Les premières puces fabriquées chez Intel utilisaient une machine appelée un «Coordinatographe» pour aider la coupe de la couche rouge (appelée couche ruby). Cependant, les positions et les longueurs des différents éléments du processeur dessinés sur le Mylar devaient être transférées à la main au coordinatographe, qui s'est fait remplacer après par une autre machine utilisant des couteaux au lieu de stylos pour plus d'efficacité et de précision. [17] De nos jours, il existe des langages de description de matériel (HDL ou Hardware Description Language en anglais) pour pouvoir décrire du matériel informatique de bas niveau (c.-à-d. une puce, un processeur). Les langages principalement utilisés sont Verilog et VHDL. Nous avons choisi d'utiliser Verilog parce qu'il est plus concis et parce que nous sommes limités par la petite fenêtre temporelle de neuf mois. [18]



FIGURE 2 – Un ingénieur dessinant un circuit sur du Rubylith

Puisque nous cherchons à faire le processeur le plus simple en raison de la limite de temps, nous avons choisi un type d'architecture RISC (Reduced Instruction Set Computer). Nous n'avions donc pas opté pour un nombre plus grand d'instructions plus complexes (donc un CISC, Complex Instruction Set Computer ou architecture de processeur à jeu d'instructions étendu, donc avec plus d'instructions complexes) en raison de notre décision de faire un processeur simple, car, généralement, plus il y a d'instructions, plus le processeur prendra de la place sur le circuit intégré. Ainsi, nous avons décidé de nous limiter à un maximum de 64 instructions.

Nous avons aussi envisagé de faire notre processeur en 64 bits pour voir à quel point il serait difficile de faire un processeur comme ceux de nos ordinateurs actuels (x86, [19] ARM [20]). Pour simplifier, le code d'instruction de notre processeur (opcode en anglais) est codé en 64 bits, dont les 6 premiers bits de poids fort (6 bits, car 64 instructions, et $6 = log_2(64)$) désignent le numéro de l'instruction choisie. La

15. ATMEL CORPORATION. *8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash*. Page consultée le 26 Mai 2024. URL : http://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf.

16. ARDUINO S.R.L. *Arduino Uno Rev3*. Page consultée le 26 Mai 2024. URL : https://store.arduino.cc/products/arduino-uno-rev3.

17. Andrew M. VOLK, Peter A. STOLL et Paul METROVICH. *Recollections of Early Chip Development at Intel*. Page consultée le 26 Mai 2024. URL : https://deramp.com/downloads/mfe_archive/050-Component%20Specifications/Intel/Recollections%20of%20Early%20Chip%20Dev.pdf.

18. DIFERR.COM. *10 Differences Between verilog and vhdl*. Page consultée le 26 Mai 2024. URL : https://diferr.com/difference-between-verilog-and-vhdl/.

19. MICROSOFT. *x86 Architecture*. Page consultée le 1er Juin 2024. URL : https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/x86-architecture.

20. ARM LIMITED. *CPU Architecture*. Page consultée le 1er Juin 2024. URL : https://www.arm.com/architecture/cpu.

longueur de notre code d'instruction est toujours de 64 bits et est donc non variable pour une architecture plus légère. Nous avons choisi de munir notre processeur avec 16 registres à usage général, même s'il y a des processeurs avec plus ou moins de registres (comme le MIPS [21] avec 32 registres ou le x86 avec 8 registres [22]). Nous avons aussi décidé de ne pas implémenter les registres spéciaux (comme le registre de pile, appelé Stack pointer en Anglais, qui indique le plus haut de la pile, utilisé pour sauvegarder des adresses dans des fonctions recursives) pour simplifier le processeur.

Pour représenter les nombres négatifs, nous avons utilisé la représentation du complément à deux (pour convertir un nombre x en un nombre négatif, il suffit d'inverser chaque chiffre binaire de ce nombre, et puis de lui ajouter 1). Cette représentation nous permet d'effectuer les mêmes opérations sur un nombre qu'il soit négatif ou pas. Nous avons décidé de ne pas implémenter les nombres à virgule flottante, [23] car, pour un processeur simple, nous n'avons pas besoin de les calculer nativement, et une telle implémentation exigera une unité à virgule flottante pour calculer le tout. Les calculs par cette unité ne se font jamais en un cycle (un cycle signifie qu'après un coup d'horloge, le résultat sera disponible à l'utilisation. Quand il y a plus qu'un seul cycle pour une instruction, on parle de pipelining, qui veut dire qu'à chaque coup d'horloge, le processeur calcule un petit peu de l'instruction et à la fin donne le résultat final [24]), mais généralement en plus que 2 cycles (par exemple, 3 cycles pour la multiplication à virgule flottante et au moins 19 cycles pour la division à virgule flottante chez le processeur Intel Pentium à 32 bits [25]) pour ne pas ralentir le processeur. Nous avons aussi décidé de ne pas implémenter ni la multiplication ni la division dans notre ALU (Arithmetic Logic Unit ou l'unité d'arithmétique et de logique, en substance la calculatrice du processeur) pour ne pas créer un pipeline de plusieurs cycles, qui nous prendrait beaucoup de temps à implémenter, et pour que l'ALU calcule tout en un seul cycle, qui simplifierait le codage.

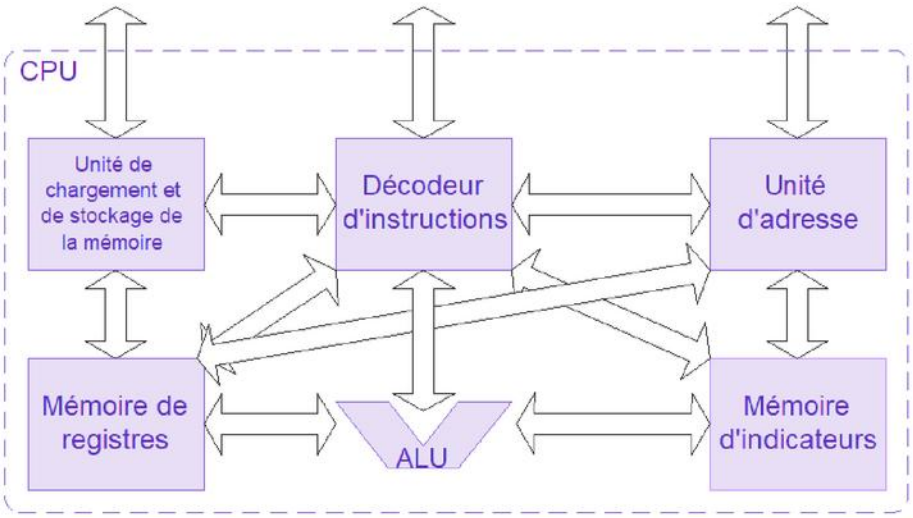


FIGURE 3 – Les unités généralement trouvées dans un processeur

3.2 Indicateurs

Nous avons choisi de munir notre processeur de 4 indicateurs (*flags* en anglais) : *Zéro*, *Négatif*, *Carry* et *Overflow*.

L'indicateur Zéro est enclenché par l'ALU quand le résultat de la dernière opération de celle-ci est nul.

L'indicateur Négatif est enclenché par l'ALU quand le résultat de la dernière opération est négatif. Pour calculer ceci, nous avons implémenté un deuxième résultat d'opération à 65 bits spécialement fait pour calculer cet indicateur. Pour calculer cet indicateur, nous regardons si le 65ème bit ou le 64ème bit de ce résultat est positif. Ce résultat est vrai, car tout nombre négatif représenté à l'aide du complément à deux a son bit de poids le plus fort enclenché à 1. En additionnant un nombre négatif avec un nombre positif, le bit du poids le plus fort reste à 1. Mais si nous additionnions un nombre négatif avec un nombre

21. Crystal CHEN, Greg NOVICK et Kirk SHIMANO. *MIPS*. Page consultée le 10 Juillet 2024. URL : https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/mips/index.html.
22. MICROSOFT, *x86 Architecture*, op. cit.
23. JEAN-MICHEL MULLER ET AL. *Handbook of Floating-Point Arithmetic*. Page consultée le 27 Mai 2024. URL : https://ieeemilestones.ethw.org/w/images/5/54/Handbook_Floating_Point_Chapter_3.pdf.
24. Crystal CHEN, Greg NOVICK et Kirk SHIMANO. *How Pipelining Works*. Page consultée le 27 Mai 2024. URL : https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/pipelining/index.html.
25. Agner FOG. *Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD, and VIA CPUs*. Page consultée le 27 Mai 2024. URL : https://www.agner.org/optimize/instruction_tables.pdf.

négatif, le 64ème bit deviendrait 0. C'est pourquoi nous avons choisi de regarder aussi le 65ème bit (Figures 4 et 5).

63			0
	1111	···	0110
	1111	···	1010
1	1111	···	0000

FIGURE 4 – Un nombre négatif additionné à un autre nombre négatif

63			0
	1111	···	0110
	0000	···	1000
0	1111	···	1110

FIGURE 5 – Un nombre négatif additionné à un nombre positif

L'indicateur Carry est enclenché quand le résultat de l'opération déborde de 64 bits, donc quand il y a eu une retenue. Pour le calculer, on regarde simplement le 65ème bit du résultat.

L'indicateur Overflow (signifie «débordement» en anglais) est positif lorsque le résultat de la dernière opération ne pourra pas être utilisé car il a débordé de l'espace des nombres positifs et est devenu négatif dans la représentation du complément à deux. Pour le calculer, nous faisons un OU exclusif entre le 65ème et le 64ème bit du résultat. Si le résultat a son 65ème et son 64ème bit à 0, alors il est positif. Si le résultat a son 65ème et son 64ème bit à 1, alors il est négatif. S'ils ne sont pas égaux, alors quelque chose s'est mal passé. Cet indicateur est donc enclenché à 1 quand le résultat de l'opération est non fiable et donc quand nous ne pouvons pas nous en servir.

3.3 Interruptions

Nous avons aussi décidé de mettre de côté les interruptions, même si presque tous les autres processeurs grand public en ont (par exemple ARM,[26] Atmega128,[27] Zilog Z80[28]), à cause d'une question de temps. Les interruptions sont en général des entrées du processeur qui, à l'aide d'un signal, le font s'arrêter momentanément et aller exécuter un programme prédéfini à l'avance. En général, l'exécution des instructions saute à une ligne prédéfinie à l'avance par le programmeur après avoir sauvegardé le contexte (les indicateurs pour pouvoir les restaurer après). Pendant que le programme de l'interruption est exécuté, toutes les autres interruptions sont bloquées. Le programme finit par une instruction disant au processeur de revenir à la ligne de laquelle il est brusquement parti et de recharger le contexte d'indicateurs, donc de changer les indicateurs. À ce moment-là, les interruptions arrêtent d'être bloquées et ce processus peut encore une fois se répéter.

3.4 Mémoire et bus de données

Pour la mémoire, nous avons décidé d'implémenter un module servant à commuter entre les différentes mémoires du processeur (comme la ROM dans laquelle il y a le programme, la RAM, les entrées ou sorties du processeur). En enclenchant le 32ème bit, le programmeur peut accéder aux diverses entrées et sorties du processeur. Ce bit a été choisi arbitrairement, mais en s'assurant que les toutes premières adresses sont libres pour la ROM, car lors du démarrage, notre processeur va commencer par lire le programme à partir l'adresse 0. Pour le son, nous avons muni le processeur de sorties PWM (Pulse Width Modulation, qui génère des signaux carrés à longueur variable. Ces signaux sont intégrés par le haut-parleur en fonction du temps pour donner un signal analogique).

Nous avons mis la mémoire du programme (celle qui contient les instructions) et la RAM dans le même bloc de mémoire, pour pouvoir exécuter des programmes directement depuis la RAM et non pas de la

26. Ahmed Fathy Mohammed ABDELRAZEK. *Exception and Interrupt Handling in ARM*. Page consultée le 27 Mai 2024. URL : http://classweb.ece.umd.edu/enee447.S2016/ARM-Documentation/ARM-Interrupts-3.pdf.
27. ATMEL CORPORATION. *8-bit Atmel Microcontroller with 128KBytes In-System Programmable Flash*. Page consultée le 27 Mai 2024. URL : https://ww1.microchip.com/downloads/en/DeviceDoc/doc2467.pdf.
28. ZILOG, INC. *Z80 CPU*. Page consultée le 26 Mai 2024. URL : https://www.zilog.com/docs/z80/um0080.pdf.

ROM. Quand le programme est dans la ROM, donc directement sur la puce, il est implémenté en utilisant un multiplexeur en fonction de l'adresse de l'instruction. Ceci utilise des tables de correspondances dans la puce. Donc plus le programme est grand, plus il y a de tables de correspondances, plus le processeur prend de la place sur la puce, et plus il est lent. En mettant le programme dans la RAM, nous pouvons non seulement nous débarrasser de ce problème, mais aussi mettre les données du programme dans la RAM (comme les pièces de jeu de Tetris, ou une police pour écrire sur l'écran), dans le même endroit que le programme.

Nous pouvons donc en venir à la conclusion que l'architecture de notre processeur est une architecture de type Harvard (Figure 6), car les bus des données et des instructions sont séparés.

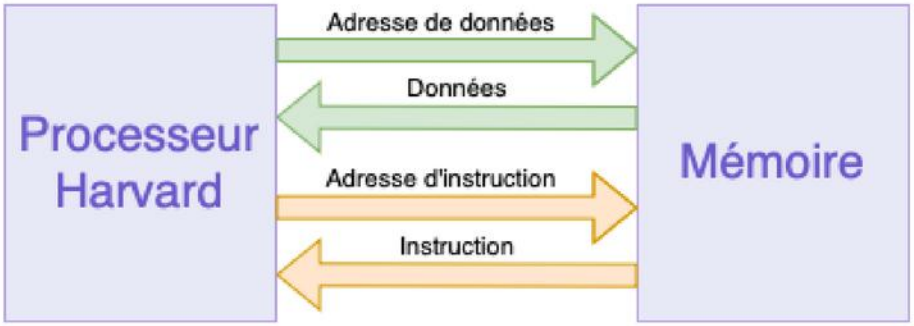


FIGURE 6 – Un processeur ayant une architecture de type Harvard avec ses bus de données

3.5 L'utilisation d'un FPGA

De nos jours, un processeur est implémenté en utilisant un circuit intégré. Pour fabriquer une puce, il faut beaucoup d'argent (plus que 10 millions de francs [29]). Limités par notre budget, nous avons donc décidé de ne pas en fabriquer une, mais de l'implémenter sur un circuit reprogrammable. En effet, nous avons utilisé un FPGA (Field Reprogrammable Gate Array), un circuit intégré dont les portes logiques peuvent être reconfigurées pour réaliser n'importe quel circuit logique, par exemple un processeur. Cependant, il faut plus de transistors par porte logique pour que celle-ci puisse être reprogrammable, c'est pourquoi les FPGA sont beaucoup plus lents qu'un circuit intégré normal, sur lequel les transistors ne peuvent pas être reconfigurés.

L'utilisation d'un FPGA est très importante pour pouvoir tester le processeur sans avoir à le refabriquer à chaque fois, et pour faire un prototype d'un circuit intégré. Quand le circuit du prototype est prêt et validé, il est alors fabriqué en silicium sur une puce. Cette méthode est utilisée par presque toutes les compagnies qui fabriquent des puces, pour éviter la perte d'argent et de temps en cas d'erreur. Un grand exemple d'une erreur serait le bug du processeur Pentium de Intel, qui se trompait dans les divisions à cause d'une erreur dans un tableau de correspondances. [30]

29. BRIAN BAILEY - SEMIENGINEERING.COM. *What Will That Chip Cost ?* Page consultée le 10 Juillet 2024. URL : https://semiengineering.com/what-will-that-chip-cost/.
30. Vaughan PRATT. *Anatomy of the Pentium Bug.* Page consultée le 10 Juillet 2024. URL : http://boole.stanford.edu/pub/anapent.pdf.

4 Fonctionnement

4.1 Lexique

Un bit est un chiffre qui peut être soit 0, soit 1. Un octet (byte, en anglais), composé de 8 bits, est donc un nombre à huit bits. Dans notre cas, un mot de notre processeur est 64 bits, soit 8 octets.

Le format hexadécimal est la représentation en base 16 d'un nombre. Par exemple, le nombre 11 en base 10 est représenté par B en base 16. Les chiffres hexadécimaux à partir de 10 sont représentés par les lettres de A à F. Ce format se traduit bien en binaire puisqu'il suffit de convertir chaque chiffre hexadécimal en 4 bits.

Un bus de données est un ensemble de fils qui permettent de transférer des données entre les différents composants d'un ordinateur. Par exemple, des données de 4 octets pourront circuler à travers un bus de données d'une largeur de 4 octets. Un bus connecté à deux unités différentes peut transférer par exemple une adresse ou des données.

4.2 Notions préliminaires

Notons d'abord que tout processeur dispose d'un compteur de programme, qui compte à partir de 0, et avance d'un en un sauf lors d'un «saut» à une autre adresse. Comme tout processeur, lors du démarrage, notre processeur reçoit un signal *RESET* pour initialiser le compteur de programme à 0, que nous avons appelé *PC* pour *program counter*, ou compteur de programme en français. Pour implémenter les sauts, nous avons doté notre processeur d'un deuxième compteur de programme que nous avons appelé *PC_NEW*. Lors du fonctionnement normal, hors saut, le compteur *PC_NEW* est fixé à la valeur du premier compteur de programme plus un. Lors d'un saut, le compteur *PC_NEW* est fixé à l'adresse du saut désiré. Finalement, à chaque coup d'horloge, le compteur de programme est mis à la valeur de *PC_NEW*.

Parlons d'abord de registres. Un registre, en général, est un très petit espace de stockage dans lequel on peut sauvegarder un nombre, ou un peu de données. Notre processeur est muni de 16 registres dont chacun est muni d'un espace de stockage de 64 bits pour que les longueurs de celui-ci et d'une instruction coïncident.

Une instruction est une chaîne de bits qui spécifie une opération à effectuer. Par exemple, cette opération peut consister en l'addition de deux registres du processeur, de comparer deux nombres afin de mettre un indicateur, ou *flag* en anglais, à 1. Cette opération peut aussi dire au processeur de continuer son exécution d'instructions à partir d'une adresse donnée (ce qui est appelé un *saut*). Cette instruction peut aussi signaler au processeur d'envoyer des données entre les différentes mémoires, comme la mémoire de registres et la mémoire RAM. Cette opération peut aussi signifier de ne rien faire du tout.

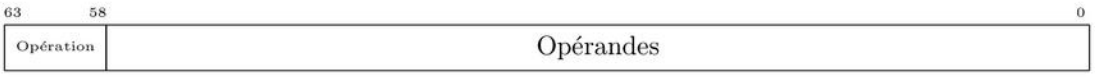


Figure 7 – *Champ de bits d'une instruction chez notre processeur*

Notons que, dans notre cas, nous avons décidé de représenter les données binaires sous forme de little endian. Cela signifie que, comme illustré sur le schéma précédent (Figure 7), les bits de poids fort sont à «gauche» de l'expression.

En effet, les 6 derniers bits de l'instruction définissent l'opération à effectuer (Figure 7). Puis, des paramètres, comme le numéro de registre, comme quelques adresses ou comme des données, seront représentés sur les bits restants. Nous avons choisi de faire les 6 derniers bits de l'instruction comme nombre de l'opération en raison du nombre de nos instructions : dans le développement, en optimisant le jeu d'instructions, nous sommes arrivés à 38 instructions. Comme $6 = \lceil log_2\ 38 \rceil$, nous avons décidé d'utiliser 6 bits pour le nombre de l'opération, comme dit dans la partie précédente. Le jeu d'instructions figure en annexe (Partie 12.5). Comme notre processeur ne s'arrête sous aucune instruction, le processeur exécute donc une instruction par coup d'horloge.

4.3 Implémentations

Notre logiciel de simulation et de synthèse, Vivado, a calculé une vitesse maximale d'horloge pour notre implémentation de 40MHz. Nous savons aussi que notre écran demande une vitesse d'horloge allant de 40MHz à 60MHz (avec une vitesse recommandée de 50MHz). C'est pourquoi la vitesse de l'horloge de notre processeur est à 40MHz.

7	6	5	4	3	2	1	0
rouge			vert			bleu	
r	r	r	v	v	v	b	b

Figure 8 – *Représentation d'une couleur dans notre VRAM*

La *RAM*, appelée aussi *mémoire vive*, est la mémoire dans laquelle seront stockés les données et le programme. La *VRAM* est la mémoire graphique, utilisée par les cartes graphiques modernes pour stocker des informations en rapport avec le calcul graphique.[31] Nous l'avons employée pour stocker l'image envoyée sur l'écran, c'est-à-dire la couleur de chaque pixel sur l'écran. Comme nous sommes limités par 225KB de blocs de mémoire dans notre puce, nous avons recouru à 128KB pour la RAM et 64KB pour la VRAM, pour arrondir aux puissances de 2. Comme nous n'avons que 64KB de VRAM, nous avons décidé de représenter chaque couleur en un seul octet (3 bits pour le rouge, 3 bits pour le vert et 2 bits pour le bleu, Figure 8). Ceci nous facilite la modification des pixels sur l'écran, parce que nous avons implémenté une instruction servant à modifier un octet dans la mémoire. Nous avons décidé de limiter la partie adressable de l'écran à des dimensions de 512 x 480 pixels réels, qui est facilement représentable en puissances de deux. Comme les blocs de mémoire (bloc RAM) les plus convenables sont sur 14 bits, ils peuvent stocker 16KB de données, et comme nous n'en avions pas assez, nous avons décidé d'en employer 4 ce qui revient à dire que nous avons décidé d'utiliser 64KB de VRAM. Mais, comme $512 \cdot 480$ est autour de 256KB, nous ne pouvions pas nous permettre de dépasser la limite de RAM installée sur la puce directement (225 KB). C'est pourquoi nous avons fait recours à des pixels deux fois plus grands. La taille logique de l'écran est donc de 256 x 240. Pour implémenter les 128KB de RAM, nous nous sommes servis de 8 unités de RAM séparées pour chaque octet, pour simplifier le stockage de mémoire par octets (pour les instructions *mov*, *mem* et *memi*, comme dit dans la partie 12.5).

4.4 Détails du fonctionnement du processeur

À chaque coup d'horloge, le processeur lit l'instruction à l'adresse du compteur de programme. Puis, cette instruction entre dans le décodeur d'instructions du processeur. Ensuite, suivant l'instruction, ce décodeur donne des signaux de contrôle aux différentes unités du processeur comme, par exemple, l'unité arithmétique et logique, l'unité d'adresse, l'unité de chargement et de stockage de la mémoire. Il contrôle aussi les indicateurs, ou *flags* en anglais, que l'instruction devrait modifier ou pas, en envoyant des signaux dans la mémoire de registres et d'indicateurs. Dans notre cas, les adresses internes des registres demandés par l'instruction seront stockées dans les bus nommés *ADDRX* et *ADDRY* pour les registres qu'il faudrait lire, et *ADDRD* pour le registre de destination. De plus, le décodeur d'instructions envoie des signaux hors du processeur pour contrôler la mémoire externe à travers l'unité de contrôle de mémoire qui se trouve hors du processeur. Cette unité de contrôle de mémoire commute entre les différentes mémoires comme la *RAM* ou comme la *VRAM* qui contient l'image qui est affichée sur l'écran dans notre cas. Le décodeur d'instructions fixe aussi l'adresse des registres source et destination pour les calculs et fixe aussi un nombre présent parfois dans l'instruction, appelé immédiat, qui nous laisse, par exemple, additionner des nombres aux registres en une seule instruction, au lieu de mettre un nombre dans un autre registre et après les additionner ensemble.

Si l'instruction reçue est une instruction arithmétique ou la modification du contenu d'un registre (ceci fait allusion à l'instruction ldi), l'ALU est celui qui va calculer le résultat, puis donner le résultat à la mémoire de registres. Pour ce faire, le décodeur d'instructions va envoyer un signal *NOP* (ne rien faire) à toutes les unités sauf à l'ALU. Dans notre cas, pour représenter *NOP* dans les signaux du processeur, nous avons choisi 0 comme signal. Lorsque l'instruction reçue demande de faire un saut, c'est l'unité d'adresse qui va faire le travail. Comme dit précédemment, la configuration des signaux sera la même (seule cette unité verra un signal qui l'obligera à faire quelque chose). Quand l'instruction exige un transfert de données, c'est l'unité de chargement et de stockage de la mémoire qui va s'en occuper. Finalement, quand l'instruction exige l'exécution d'une opération qui va altérer la mémoire d'un registre, l'indicateur interne nommé *flag_write_enable* va être activé, pour montrer à la mémoire de registres de modifier le registre à l'adresse *ADDRD* à l'aide du résultat quelle reçoit.

4.5 Parties du processeur

Ci-dessous (Figure 9), nous avons représenté les connexions des divers composants du processeur. Nous allons maintenant décrire chacun des modules en détail. Les signaux représentés en orange sont des signaux d'adresse. Les signaux coloriés en bleu sont des signaux de contrôle, tandis que les signaux

31. Alan Bradley, PC MAG. *What Is VRAM ? Why Games Are Using it More, and How to Optimize Your PC*. Page consultée le 3 Juin 2024. URL : https://www.pcmag.com/how-to/what-is-vram-why-games-are-using-it-more-and-how-to-optimize-your-pc.

coloriés en rose sont des signaux de contrôle de longueur de un bit. Les signaux représentés en vert sont des données brutes, et les signaux en noir sont les signaux de contrôle global, donc uniquement l'horloge (clk) et la réinitialisation (rst).

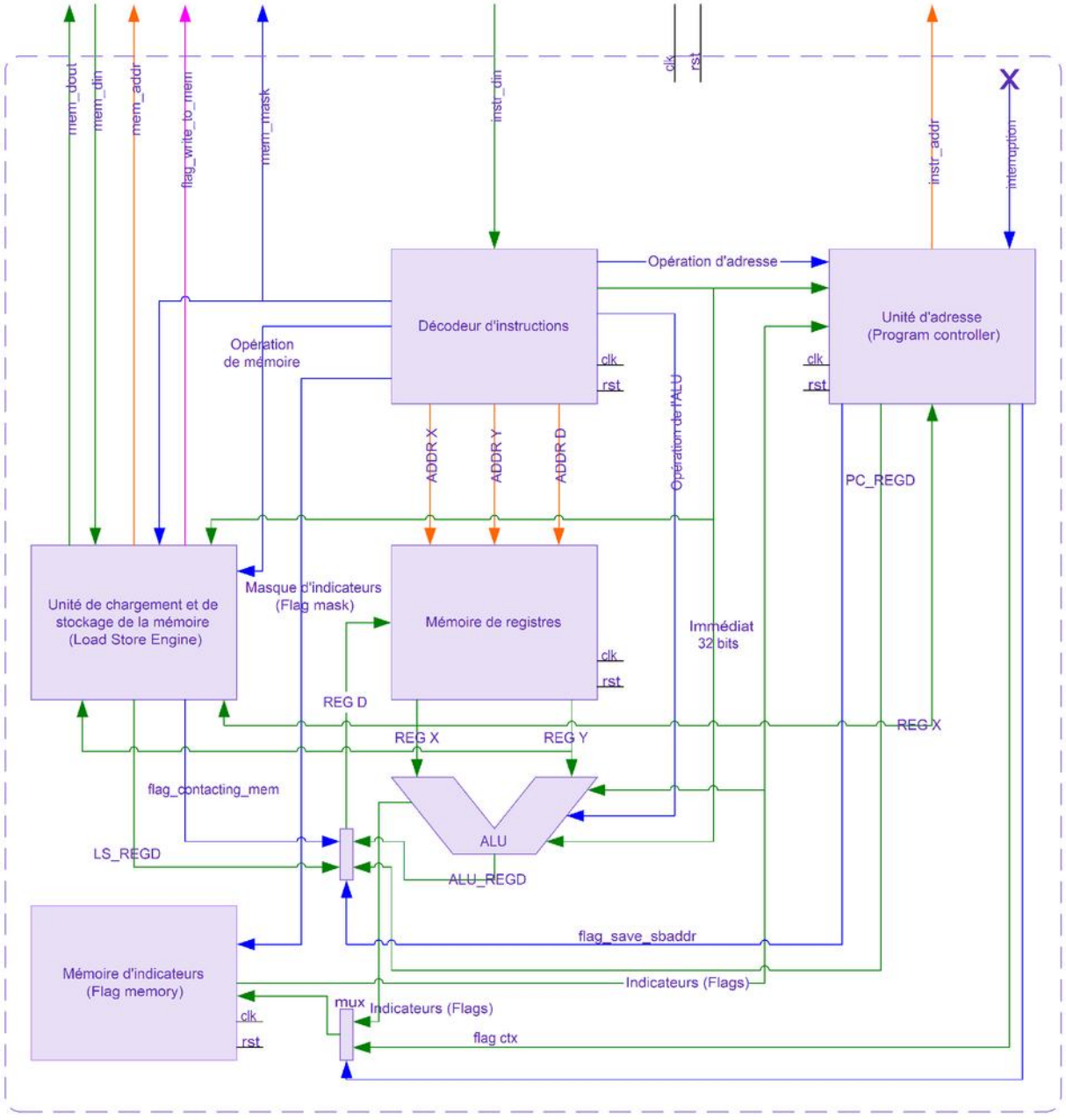


Figure 9 – *Ordinogramme de l'intérieur de notre processeur*

Comme expliqué plus haut, le décodeur d'instructions envoie des signaux de contrôle aux différentes unités du processeur en fonction de l'instruction reçue (en bleu).

La mémoire de registres reçoit les adresses de source et destination de registres, dont l'adresse du registre Rs_1 est nommée *ADDR X* sur le schéma et l'adresse du registre Rs_2 est représentée comme *ADDR Y*. L'adresse *Rd* est nommée *ADDR D* sur le schéma. Ce qui est contenu aux adresses *ADDR X* et *ADDR Y*, donc *REG X* et *REG Y*, rentre dans l'ALU pour pouvoir faire des calculs dessus (par exemple, pour les additionner). *REG X* et *REG Y* rentrent aussi dans l'unité qui s'occupe de la mémoire externe, pour être ensuite, en fonction de l'opération de contrôle (signal nommé «Opération de mémoire» sur le diagramme), envoyés dans la mémoire après modification. *REG X* est aussi envoyé dans l'unité d'adresse pour pouvoir faire des sauts. L'unité de contrôle de mémoire décide en fonction du signal de contrôle reçu s'il faudrait enregistrer *LS_REGD* ou *ALU REGD* en modifiant *flag_contacting_mem*. Le décodeur d'instructions décide s'il faudrait enregistrer le résultat *REG D* ou pas.

Un immédiat long de 32 bits rentre dans l'ALU et dans l'unité d'adresse pour pouvoir faire de diverses opérations en utilisant directement un nombre, comme mentionné dans la description du jeu d'instructions (voir Annexes, chapitre 12.4)

D'abord, pour stocker les indicateurs, le décodeur d'instructions décide s'il est nécessaire de le faire pour chacun des indicateurs ou pas (régulé par le signal *Masque d'indicateurs*). Puis, l'unité d'adresse choisit le résultat à enregistrer (qu'il soit le sien ou celui de l'ALU). Généralement, l'unité d'adresse ne demande l'enregistrement de sa version des indicateurs (nommée *flag ctx*) que lorsque le décodeur d'instructions reçoit l'instruction de rétablissement des indicateurs (après la fin de l'exécution d'un sous-programme, par exemple).

Nous avons aussi indiqué l'emplacement du signal d'interruption, si nous l'avions implémenté.

Chaque unité non combinatoire (donc une unité qui ne génère que des signaux de contrôle, comme le décodeur d'instructions, par exemple) reçoit un signal d'horloge et de réinitialisation pour pouvoir stocker des informations.

Enfin, voici la description des divers signaux sortant et rentrant dans le processeur. *mem_dout* est le signal sortant vers la mémoire externe comportant les données que le processeur souhaite écrire à l'adresse *mem_addr*. *mem_din* est le signal entrant dans le processeur qui comporte les données à l'adresse *mem_addr*. Le signal *flag_write_to_mem* donne le feu vert à la mémoire d'inscrire *mem_dout* à l'adresse *mem_addr*. Le signal *instr_din* contient l'instruction à l'adresse *instr_addr*.

Le processeur est connecté au contrôleur de RAM (comme montré sur la Figure 10), au contrôleur d'IO, donc celui qui s'occupe des entrées et sorties du système sur puce (donc du Cmod A7 que nous utilisons) et au contrôleur d'écran, qui génère les signaux nécessaires pour le bon fonctionnement de l'écran. Pour commuter entre les différentes unités, nous avons mis deux multiplexeurs appelés *mux1* et *mux2* sur le schéma. Ces derniers ne sont pas nécessairement des modules Verilog dans le code source du système, mais sont directement implémentés dans le fichier `wrapper.v`. Pour commuter entre les différentes unités, nous avons décidé de faire usage de l'adresse comme sélecteur d'unité. Si le 32ème bit est mis à 1, alors le contrôleur d'IO est sélectionné, c'est-à-dire qu'une opération de stockage se traduira par la modification des sorties du système. Si le 64ème bit est enclenché, alors le contrôleur de l'écran sera sélectionné. Une opération de stockage se traduira donc par l'écriture dans la VRAM, donc par la modification de pixels sur l'écran.

Pour simplifier, voici les formules de calcul :

$$ram_write = (mem_write \wedge \overline{mem_addr_{32}}) \wedge (\overline{mem_addr_{64}})$$

$$io_write = (mem_write \wedge mem_addr_{32}) \wedge (\overline{mem_addr_{64}})$$

$$vram_write = mem_write \wedge mem_addr_{64}$$

Ces signaux donnent le feu vert à l'unité respective d'enregistrer des données. Notons que, dans ces formules, comme dans le code, *mem_write* correspond au signal *flag_write_to_mem*. Voici la formule des signaux qui choisissent depuis quelle unité il faudrait lire les données (la VRAM, représentant les pixels sur l'écran, est en écriture seule) :

$$ram_read = \overline{mem_write} \wedge \overline{mem_addr_{32}}$$

$$io_read = \overline{mem_write} \wedge mem_addr_{32}$$

Nous utilisons une boucle à verrouillage de phase (PLL, ou Phase-locked loop) pour générer l'horloge et le signal de réinitialisation (*rst*). Le signal *clk* est l'horloge du système, et le signal *rst* est le signal de réinitialisation du processeur. Le signal *rst* est mis à 1 pendant quelques cycles d'horloge, puis repasse à 0. Cela permet de réinitialiser les registres, et les compteurs pour remettre le processeur dans un état stable.

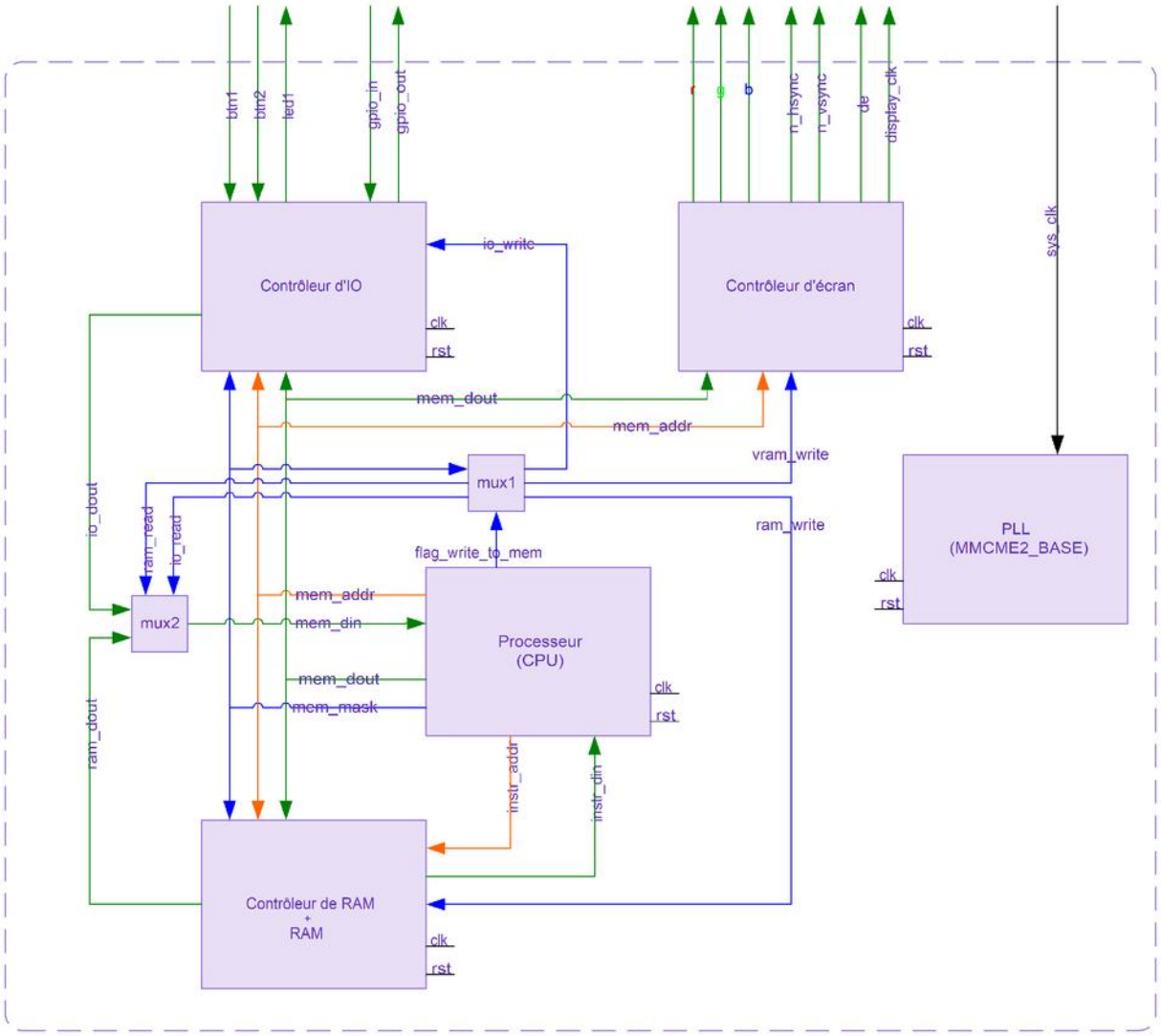


Figure 10 – *Ordinogramme du branchement de notre processeur aux différents périphériques*

De la même façon que pour le diagramme précédent, nous allons décrire les signaux entrants et sortants du système. *btn1* est la valeur du bouton 1 sur le circuit Cmod A7. Quand l'utilisateur appuie sur le bouton 1, une valeur de 1 peut être lue depuis *btn1*. Même chose pour *btn2*. Quand *led1* est mis à 1, alors la LED 1 sur le circuit Cmod A7 s'allume, sinon s'éteint. *gpio_in* est composé de 8 bits et correspond aux broches 33 à 40 inclus (par exemple, le premier bit désigne la broche numéro 33) du Cmod A7. *gpio_out* est aussi composé de 8 bits et correspond aux broches 41 à 46 inclus du Cmod A7. Les deux derniers bits de *gpio_out* ne sont pas utilisés. Les broches 47 et 48 sont l'inverse l'une de l'autre et sont des sorties PWM (Pulse Width Modulation, modulation de largeur d'impulsion), dont la résolution est de 8 bits. Ceci veut dire que nous pouvons envoyer des valeurs de 0 à 255 pour contrôler la largeur de l'impulsion PWM. Comme la vitesse de l'horloge de notre processeur est de 40MHz, la fréquence de la PWM est de $40\text{MHz}/256 = 156.25\text{kHz}$.

Le processeur pourra lire *gpio_in* et modifier *gpio_out* à travers le contrôleur d'IO. Voici le tableau de correspondances entre périphériques externes et ce que le processeur devra mettre dans la mémoire, qui s'appliquera de la même façon à *mem_din* et *mem_dout* :

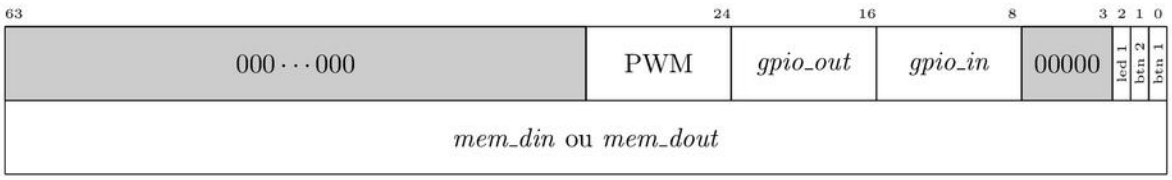


Figure 11 – *Champ de bits entre mem_din, mem_dout et les périphériques externes*

Puis, *r*, *g*, et *b*, contrôlent la couleur d'un pixel sur l'écran. Pour autoriser cette modification, le signal *de* est enclenché à 1. Pour envoyer une image sur l'écran, *hsync* est enclenché à 1 lors de transmission de données ligne par ligne. Avant et après chaque envoi de données pour une ligne, il y a une courte pause pendant laquelle *hsync* est mis à 0. Pendant la transmission de lignes, *vsync* est enclenché à 1. Aussi, avant et après la transmission de toutes les lignes, *vsync* est mis à 0. Enfin, *display_clk* est connecté simplement à *clk*, car, comme mentionné au début du sous-chapitre 4.3, la vitesse de notre horloge correspond à celle de notre écran.

Enfin, *sys_clk* est utilisé pour contrôler la boucle à verrouillage de phase (PLL) qui va générer l'horloge principale (*clk*).

4.6 Prototype

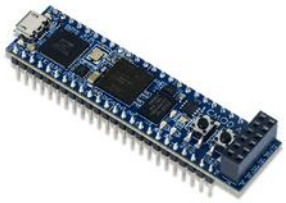


Figure 12 – *Cmod A7, le module sur lequel nous avons réalisé notre processeur*

Pour implémenter le tout, nous avons décidé d'utiliser un circuit imprimé fait par *Digilent*,[32] qui abrite un FPGA de *Xilinx*.[33] Pour envoyer la configuration de portes logiques sur notre puce, le FPGA, nous avons utilisé *Vivado*,[34] un logiciel de *Xilinx, AMD*. En effet, nous avons choisi donc le *Digilent Cmod A7*[35] (Figure 12), avec la puce *Xilinx ARTIX-7 35T cpg236-1*[36] parce que ce circuit imprimé n'était pas très cher (autour de 90 CHF) (Figure 12).

32. Digilent. *Digilent – Start Smart, Build Brilliant*. Page consultée le 1er Juin 2024. URL : https://digilent.com/.
33. Advanced Micro Devices, Inc. *Adaptive SoCs and FPGAs*. Page consultée le 31 Mai 2024. URL : https://www.amd.com/en/products/adaptive-socs-and-fpgas.html.
34. Advanced Micro Devices, Inc. *Vivado Overview*. Page consultée le 31 Mai 2024. URL : https://www.xilinx.com/products/design-tools/vivado.html.
35. Digilent. *Cmod A7-35T : Breadboardable Artix-7 FPGA Module*. Page consultée le 31 Mai 2024. URL : https://digilent.com/shop/cmod-a7-35t-breadboardable-artix-7-fpga-module/.
36. Advanced Micro Devices, Inc. *AMD Artix™ 7 FPGAs*. Page consultée le 31 Mai 2024. URL : https://www.amd.com/en/products/adaptive-socs-and-fpgas/fpga/artix-7.html.

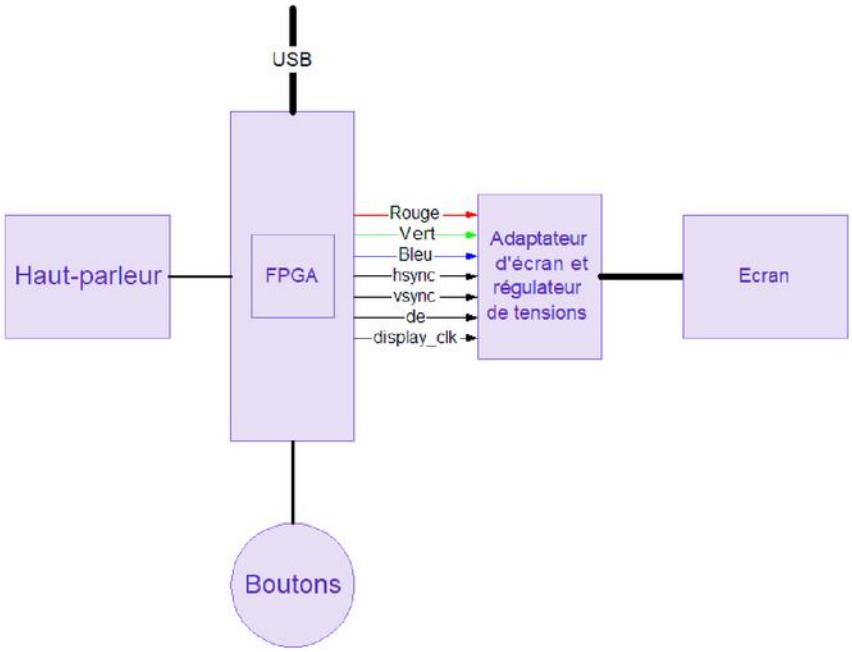


Figure 13 – *Ordinogramme de notre prototype*

Pour le connecter à l'écran, nous avons utilisé un adaptateur d'écran ayant un convertisseur de tensions (Figure 13) que nous avons connecté au *Digilent Cmod A7* (Figure 14). Pour le son, nous avons connecté un haut-parleur au *Digilent Cmod A7*. Pour l'alimentation, nous avons utilisé un câble USB. Pour interagir avec notre système, nous avons utilisé 4 boutons poussoirs ayant 4 LEDs intégrées. L'écran,[37] le convertisseur de tensions[38] et les boutons[39] étaient achetés chez *Adafruit*. Le haut-parleur[40] était acheté chez *Mouser*.

Dans notre implémentation de Tetris, les boutons sont connectés aux 4 premières broches de GPIO «entrée» (broches 33,34,35 et 36) dont les LEDs sont connectées aux 4 premières broches de GPIO «sortie» (qui correspondent aux broches 41,42,43 et 44). Les deux broches du haut-parleur sont connectées aux deux dernières broches de GPIO «sortie» (broches 47 et 48). Les signaux *h_sync* et *v_sync* sont connectés aux broches 1 et 2 respectivement. Les signaux *de* et *clk* sortent des broches 3 et 4 respectivement. Le bus «rouge» sort des broches 5 à 12, le bus «vert» sort des broches 13,14, et des broches 17-22 (les broches 15 et 16 ont été sautées puisqu'elles sont des entrées analogiques). Le bus «bleu» sort des broches 23,26, et des broches 27-32 (les broches 24 et 25 ont été sautées puisqu'elles sont des broches d'alimentation).

37. Adafruit. *7.0" 40-pin TFT Display - 800x480 without Touchscreen*. Page consultée le 15 Juillet 2024. URL : https://adafru.it/2353.
38. Adafruit. *40-pin TFT Friend - FPC Breakout with LED Backlight Driver*. Page consultée le 15 Juillet 2024. URL : https://adafru.it/1932.
39. Adafruit. *Mini LED Arcade Button - 24mm Translucent Clear*. Page consultée le 15 Juillet 2024. URL : https://adafru.it/3429.
40. Mouser Electronics, Inc. *GF0876 CUI Devices*. Page consultée le 15 Juillet 2024. URL : https://www.mouser.ch/ProductDetail/CUI-Devices/GF0876?qs=WyjlAZoYn5229XpWUmpA%252Bg%3D%3D.

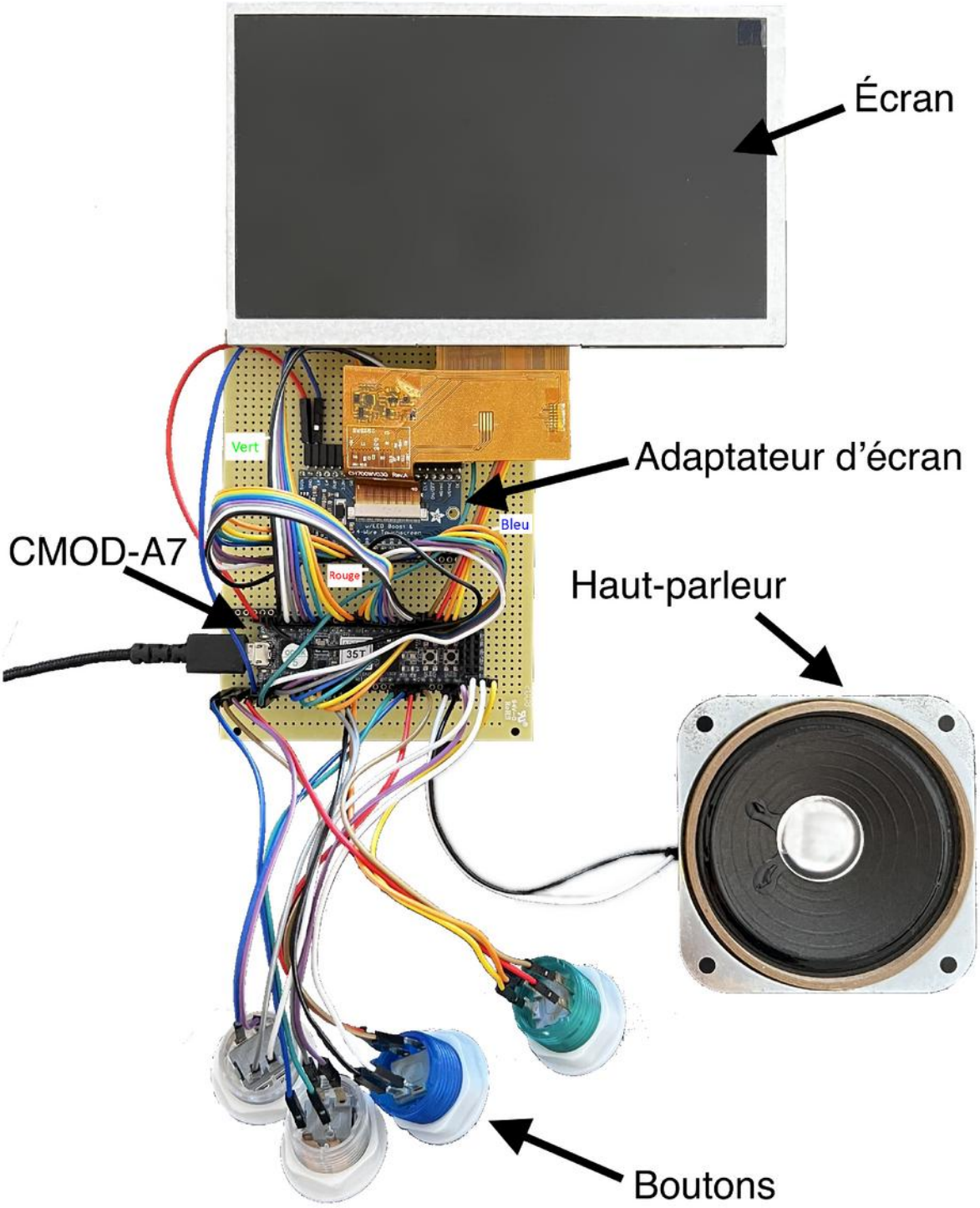


Figure 14 – *Notre prototype qui est un ordinateur complet*

4.7 Quelques détails

Pour déboguer notre processeur, nous avons implémenté un *banc de test* (*test bench* en anglais), donc un programme qui simule les différentes entrées et sorties requises par le processeur pour fonctionner. Puis, dans *Vivado*, pour voir les signaux internes du processeur, nous l'avons simulé (Figure 15).

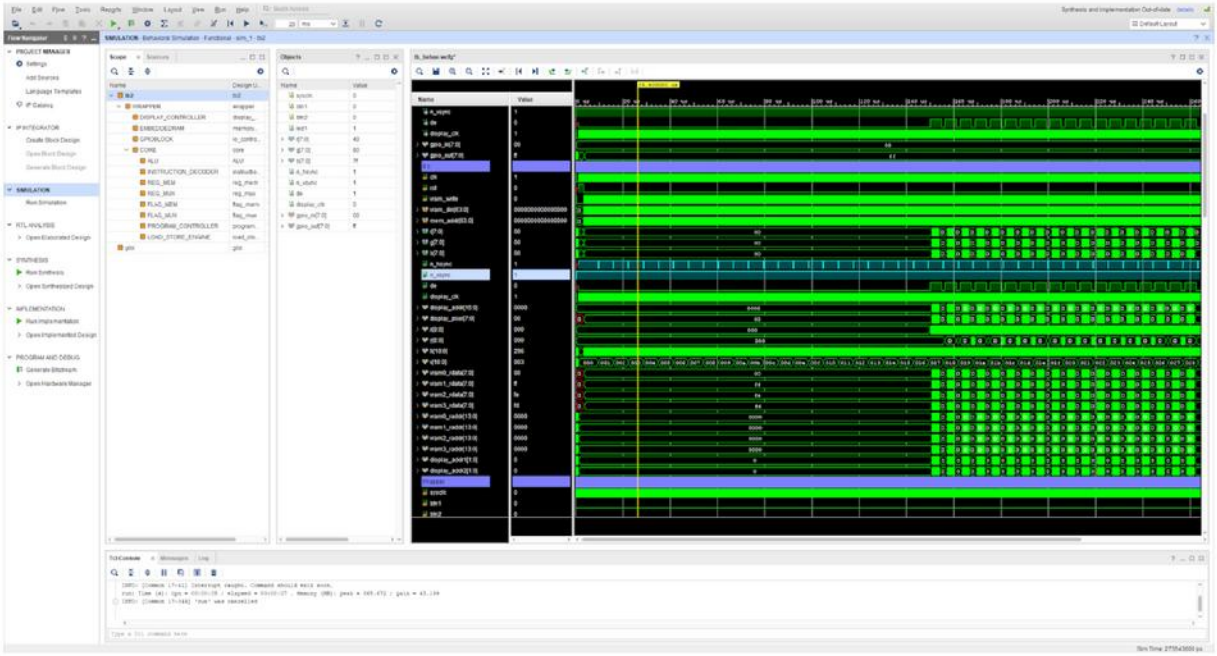


Figure 15 – *Une capture d'écran du logiciel Vivado après simulation. Les signaux n_hsync et n_vsync qui servent pour synchroniser l'envoi des données à l'écran figurent en bleu clair. Le logiciel nous a permis de déboguer le timing de ceux-ci.*

5 Programmation du compilateur d'assembleur

5.1 Introduction de l'assembleur

La plupart des programmeurs créent du code dans des langages de haut niveau, donc des langages construits pour être facilement compréhensibles par l'Homme (comme Python ou C, par exemple). Ce que la machine comprend est généralement un langage de bas niveau, comme dans notre cas. Notre processeur accepte des instructions binaires (aussi appelé *langage machine*) et pas du langage de haut niveau. C'est pourquoi, pour faciliter la tâche d'écriture de programmes pour notre processeur, nous avons conçu un compilateur, c'est-à-dire un programme qui va traduire un programme écrit en assembleur, qui est un langage de très bas niveau [41], en langage machine.

Notre compilateur traduit du code d'assembleur (Figure 16), en langage machine, donc en code binaire. Notons que chaque instruction d'assembleur correspond directement à une instruction binaire en langage machine.

Le format de l'assembleur est le suivant :

xxx : définit une étiquette au nom de *xxx*

;xxx définit un commentaire et est ignoré par le compilateur

**$**$r_x$ est le registre numéro x

x'hy est le nombre y en format hexadécimal, encodé en x bits

x'dy est le nombre y en format décimal, encodé en x bits

x'by est le nombre y en format binaire, encodé en x bits

#define a b définit a comme b.

@a est remplacé par la définition de a. Si la commande ci-dessus était détectée, alors «@a» serait remplacé par «b».

dw utilise le format d'une instruction normale, sauf qu'elle en est une pour le compilateur et pas pour le processeur. Cette commande accepte 8 bytes de données et est utilisée pour définir celles-ci. Nous avons choisi de l'appeler «*dw*» pour *define word* en anglais.

ds marche comme *dw*, sauf qu'elle n'accepte qu'une chaîne de caractères de 8 bytes. Nous avons choisi de l'appeler «*ds*» pour *define string*.

41. L'assembleur, spécifique à chaque processeur, est un langage qui se rapproche fortement du langage machine. Il est néanmoins plus compréhensible pour les programmeurs que ce dernier.

```
main:
    ldi $r0 32'b0 1'b0
    ldi $r0 32'b0 1'b1
    ldi $r1 32'b0 1'b0
    ldi $r1 32'b0 1'b1
    ldi $r2 32'd512 1'b0
    ldi $r2 32'b0 1'b1
loop:
    addi $r0 $r0 32'd1 4'b0
    cpi $r0 32'h7FFFF
    bifr 32'hA0000003 ;jump across jmpi
    nop
    jmpi loop
    nop
blink:
    nop
    ldi $r0 32'b0 1'b0
    ldi $r0 32'b0 1'b1
    xori $r1 $r1 32'hFFFFFFFF 1'b0 ; toggle led
    mem $r2 $r1 32'd0 8'b11111111 ; upload new led status to pin
    jmpi loop
    nop
```

Figure 16 – *Exemple de code en assembleur. En exécutant ce code, le processeur clignote avec la LED du bouton externe connecté aux premières broches d'entrée et de sortie du circuit Cmod A7.*

5.2 Fonctionnement du compilateur

Pour parvenir à ce but, nous avons codé en Python un script permettant de compiler du code écrit en assembleur en langage machine. D'abord, les commentaires et les espaces blancs sont enlevés complètement du programme. Ensuite, une liste est initialisée avec chaque instruction, ligne par ligne. Puis, le script Python fait un premier passage, cherchant une macro (une abréviation définie par l'utilisateur avec le mot-clé *#define*) et des étiquettes (donc pour créer des alias d'adresses, pour par exemple implémenter une fonction). En trouvant ces derniers, il stocke l'adresse de chacun d'eux dans un dictionnaire. Puis, il fait un deuxième passage pour détecter les instructions et leurs adresses. En faisant cela, il recalcule les adresses de chaque instruction, parce qu'une étiquette ne compte pas pour adresse (une étiquette est uniquement une indication au compilateur). Pour chaque instruction trouvée, il la convertit en format binaire en fonction des arguments de celle-ci. Les arguments qui désignent des étiquettes sont remplacés par les adresses calculées pour celles-ci. Toutes ces nouvelles instructions en format binaire sont stockées dans une liste et, finalement, écrites dans 8 fichiers en format hexadécimal pour après être enregistrées dans les 8 modules de RAM en allumant le système (nous avons implémenté 8 unités de RAM, comme décrit dans la partie 4.3).

6 Programmation de Tetris

Pour programmer Tetris, il nous a fallu en premier lieu inventer un algorithme. C'est pourquoi nous l'avons d'abord écrit dans le langage de programmation C, parce que celui-ci est plus facile à déboguer que l'assembleur (nous pouvons par exemple imprimer des variables dans la console ou voir en direct les valeurs des variables). C est un langage qui se compile en assembleur pour ensuite être exécuté par le processeur. [42] Dans ce langage, nous avons implémenté l'algorithme pour le jeu Tetris.

Préliminairement, parlons de ce qu'est Tetris. Dans les années 1980 en l'Union soviétique, Aleksey Pajitnov, un chercheur dans le domaine de l'intelligence artificielle et dans le domaine de la reconnaissance vocale informatique, a développé le jeu légendaire Tetris. C'est un jeu composé de tetrominos (pièces de jeu qui consistent en quatre cases) et d'un rectangle mesurant 10x20 cases. Chaque pièce descend petit à petit du haut du rectangle, et pendant cette descente, le joueur peut contrôler la pièce en la faisant pivoter ou en la bougeant à gauche ou à droite. La pièce s'arrête quand elle s'empile sur une autre ou quand elle atteint le bout du rectangle. Le joueur peut accélérer les pièces, mais ne peut ni la ralentir ni l'arrêter. Donc l'objectif du jeu est d'empiler les tetrominos si bien qu'ils forment des lignes horizontales. Quand une ligne est remplie, elle disparaît et tout ce qui était en dessus de la ligne tombe d'une case. Ce faisant, le jeu accorde des points au joueur. [43]

6.1 Programmation en C

Pour après retranscrire le code en assembleur, nous avons écrit le code de la manière la plus simple possible. Nous avons donc résolu d'utiliser une liste de 200 valeurs ($10 \cdot 20 = 200$) pour représenter le rectangle qui abrite les tetrominos dans le jeu. Afin de faire pivoter les tetrominos, la décision d'inclure un pointeur au prochain état de rotation du tetromino était prise. Notons que chaque tetromino est composé de 4 cases, et qu'il a une couleur unique. Aussi, avoir une origine géométrique pour chaque tetromino nous a permis de le déplacer plus facilement (il suffisait d'incrémenter la valeur x ou y). Avec cette origine, nous avons employé des décalages pour chaque case du tetromino. Donc nous avons utilisé la structure de données suivante : `(rotation_suivante*, sx1, sy1, sx2, sy2, sx3, sy3, couleur)`. Avec cette structure de données, pour faire pivoter le tetromino, il suffit de remplacer le pointeur du tetromino actuel par le pointeur nommé *rotation_suivante*.

Dans Tetris, il existe 7 tetrominos possibles (Figure 17), avec 4 rotations au maximum. Nous avons décidé d'omettre les états semblables pour que le programme prenne moins de place (après rotation, la pièce pourrait être la même que celle rencontrée avant). Dans la figure ci-contre (Figure 17), nous avons représenté l'origine géométrique de chaque tetromino dans une couleur bien visible.

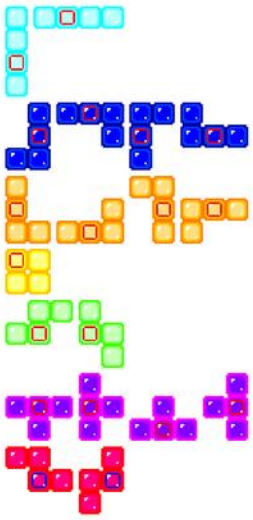


Figure 17 – *Les différents types de tetrominos : I, O, T, J, L, S, Z (de haut en bas)*

Pour pouvoir ouvrir une fenêtre et dessiner dessus, nous avons utilisé la librairie SDL de C, [44] que nous n'allons pas retranscrire en assembleur.

À chaque fois que le délai est écoulé ou qu'un bouton a été appuyé, la fonction `step` est exécutée. Dans cette fonction se trouve notre algorithme principal de Tetris. Quand elle est appelée, elle enlève le tetromino contrôlable de l'écran. Puis, elle regarde si un bouton était appuyé. Si c'était le bouton pour pivoter la pièce, elle contrôle si la pièce peut subir une rotation (en regardant si une case du nouvel état de la pièce toucherait une bordure ou une autre pièce). Si c'est le cas, elle change le pointeur de la pièce vers son état de rotation prochain. Après avoir fait cela, elle replace la pièce sur l'écran (qu'elle ait changé d'état ou pas).

Si le joueur voulait déplacer la pièce à gauche, elle vérifie de la même façon si la pièce toucherait quelque chose. Si ce n'est pas le cas, elle décrémente la coordonnée tx du tetromino. Puis elle replace celui-ci sur l'écran (qu'il ait subi un changement ou pas). De même pour le cas à droite (sauf en incrémentant tx).

Si le joueur souhaitait faire tomber le tetromino, elle regarde dans une boucle, après avoir incrémenté la coordonnée ty, si celle-ci toucherait quelque chose. Si c'est le cas, alors elle replace la pièce sur l'écran (cette fois, comme les coordonnées seront changées, éternellement), puis change les coordonnées tx et ty

42. CODECADEMY.COM. *Compiling.* Page consultée le 23 Juin 2024. URL : https://www.codecademy.com/resources/docs/c/compiling.

43. SEMENOV SERGEI - HISTRF.RU. *How a Soviet programmer made the worlds most popular game.* Page consultée le 23 Juin 2024. *Titre traduit.* URL : https://histrf.ru/read/articles/skaz-pro-to-kak-sovetskiy-programmist-samuyu-populyarnuyu-igru-v-mire-izobrel.

44. LIBSDL-ORG. *Simple DirectMedia Layer (SDL) Version 3.0.* Page consultée le 23 Juin 2024. URL : https://github.com/libsdl-org/SDL.

à $(tx, ty) = (5, 0)$. Enfin, elle choisit au hasard un nouveau tetromino, puis place son pointeur dans la mémoire dans la variable *current* qui désigne le tetromino actuel.

Si le joueur n'avait rien appuyé, la fonction `step` examine si la pièce pourrait être descendue d'une case vers le bas. Si elle peut, elle incrémente la coordonnée *ty* en dessinant le tetromino sur l'écran. Si ce n'est pas le cas, elle le place sur l'écran indéfiniment (pour la raison identique que plus haut), puis recentre les coordonnées *tx* et *ty*, et choisit un nouveau tetromino (même algorithme que pour «faire tomber la pièce»).

Enfin, si une ligne a été complétée, la fonction décale tout le rectangle vers le bas d'une case. Si nous avions implémenté un système de points en C, nous aurions la possibilité à ce moment-là d'octroyer des points au joueur.

Ce programme en C ne comportait que l'algorithme de base de Tetris. C'est pourquoi, dans l'image ci-dessous (Figure 18), il n'y avait rien d'autre que le rectangle 10x20.

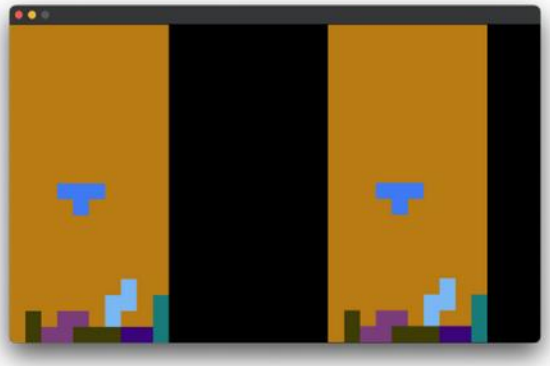


Figure 18 – *Une capture d'écran de Tetris programmé en C.*

6.2 Retranscription de l'algorithme en assembleur

Certaines parties de l'algorithme écrit en assembleur diffèrent de celui écrit en C. Par exemple, pour vérifier s'il y a une collision, celui en C vérifie pour chaque case du tetromino si elle touche une autre case du rectangle. En assembleur, nous avons décidé de copier le rectangle dans une autre partie de la mémoire, appelée *rectangle_copy*. Ensuite, nous avons enlevé les cases du tetromino dans *rectangle_copy*. Enfin, nous avons vérifié si *rectangle_copy* a changé en étant modifié par le tetromino. Si c'est le cas, alors il y a une collision.

L'algorithme de Tetris écrit en assembleur est représenté ci-dessous (Figures 19, 20 et 21).

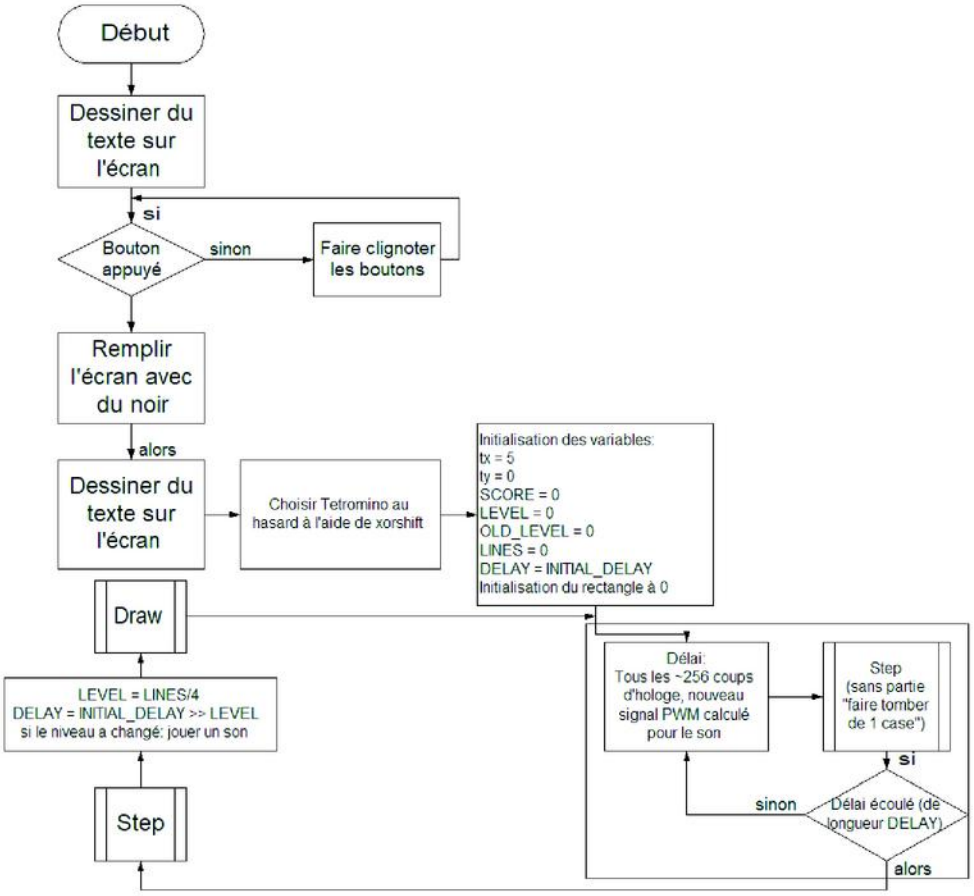


Figure 19 – *L'algorithme de Tetris écrit en assembleur.*

La variable *SCORE* est incrémentée de 25 à chaque fois qu'une ligne est complétée. Quand le joueur appuie sur «Faire tomber», alors la variable *SCORE* est incrémentée de 1 à chaque case que le tetromino passe en tombant. *LINES* est incrémentée de 1 à chaque fois qu'une ligne est complétée. La variable *DELAY* est divisée par 2 à chaque fois que *LINES* est un multiple de 4. Nous aimerons convertir le score représenté en complément à deux en texte pour l'afficher sur l'écran en n'utilisant aucune opération compliquée. Pour ce faire, nous allons manipuler les octets pour qu'ils contiennent une valeur de 0 à 9. Cette forme de stockage de nombres s'appelle «décimal codé binaire» (abbrevié DCB). Notons que cette méthode de correction ne marche que lors d'une addition de deux nombres en format DCB. Cette approche a été choisie à la place d'une autre puisqu'elle ne nécessite aucune multiplication ni division qui pourrait ralentir le programme.

La correction se présente comme suit : nous convertissons d'abord le nombre en format DCB (par exemple, 0000_2 serait converti en «0», 0101_2 serait converti en «5»). Ensuite, lorsqu'un symbole hexadécimal (le format hexadécimal représente les nombres en base 16. Par exemple, $01010010_2 = 52_{16}$ et $01110011_2 = 73_{16}$) est plus grand que 9, nous ajoutons 6 à celui-ci pour obtenir le chiffre correspondant en base 10, et ainsi de suite pour les symboles de poids plus forts. Le score total est sauvegardé dans le format hexadécimal pour pouvoir être ensuite affiché directement sur l'écran. Chaque incrémentation du score nécessite donc une correction de ce type, comme illustré dans les exemples suivants.

À titre d'illustration, si le score initial était 19_{16} et que l'on souhaite ajouter 1, nous aurons $19_{16}+1_{16} = 1A_{16}$. Par contre, comme le premier symbole est plus grand que 9, nous ajoutons 6 pour corriger la somme. Le calcul devient donc $1A_{16} + 6_{16} = 20_{16}$, ce qui est bien la somme entre 19 et 1 dans le système décimal ($19_{10} + 1_{10} = 20_{10}$).

Un autre exemple serait 99_{16}, le score actuel, auquel nous voudrions rajouter 1_{16}. On aura $99_{16}+1_{16} = 9A_{16}$. Le premier symbole étant plus grand que 9, nous ajoutons 6 comme dans l'exemple précédent. Le calcul devient $9A_{16} + 6_{16} = A0_{16}$. Puis, nous effectuons la même opération pour le deuxième symbole puisqu'il est plus grand que 9. Nous allons donc ajouter la correction de 60_{16} pour modifier le deuxième symbole. On a $A0_{16} + 60_{16} = 100_{16}$, qui est la somme corrigée attendue pour $99_{10} + 1_{10} = 100_{10}$. Ce cycle continue pour chaque symbole étant plus grand que 9 dans le nombre jusqu'au dernier symbole.

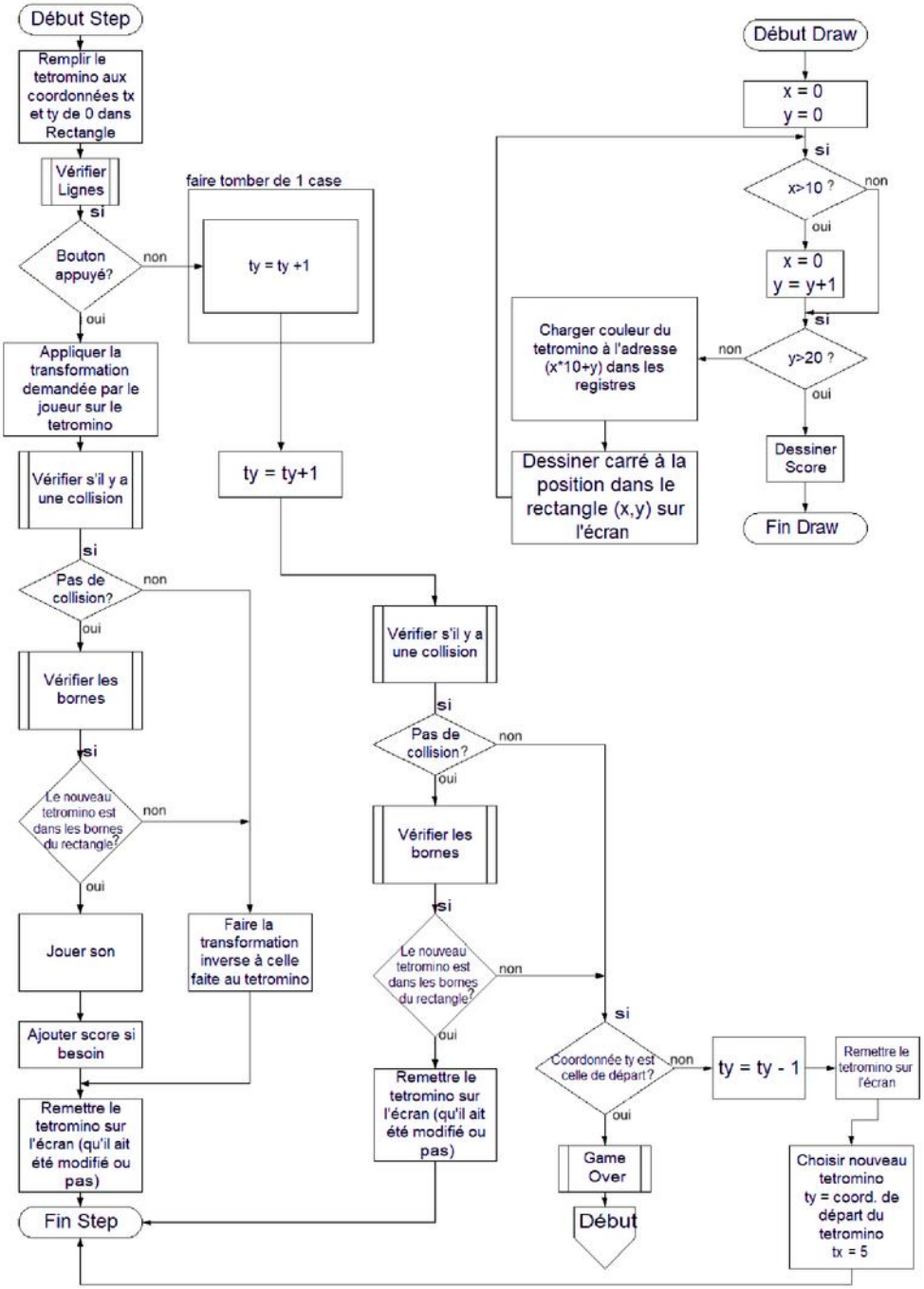


Figure 20 – *Algorithme de la fonction step et draw.*

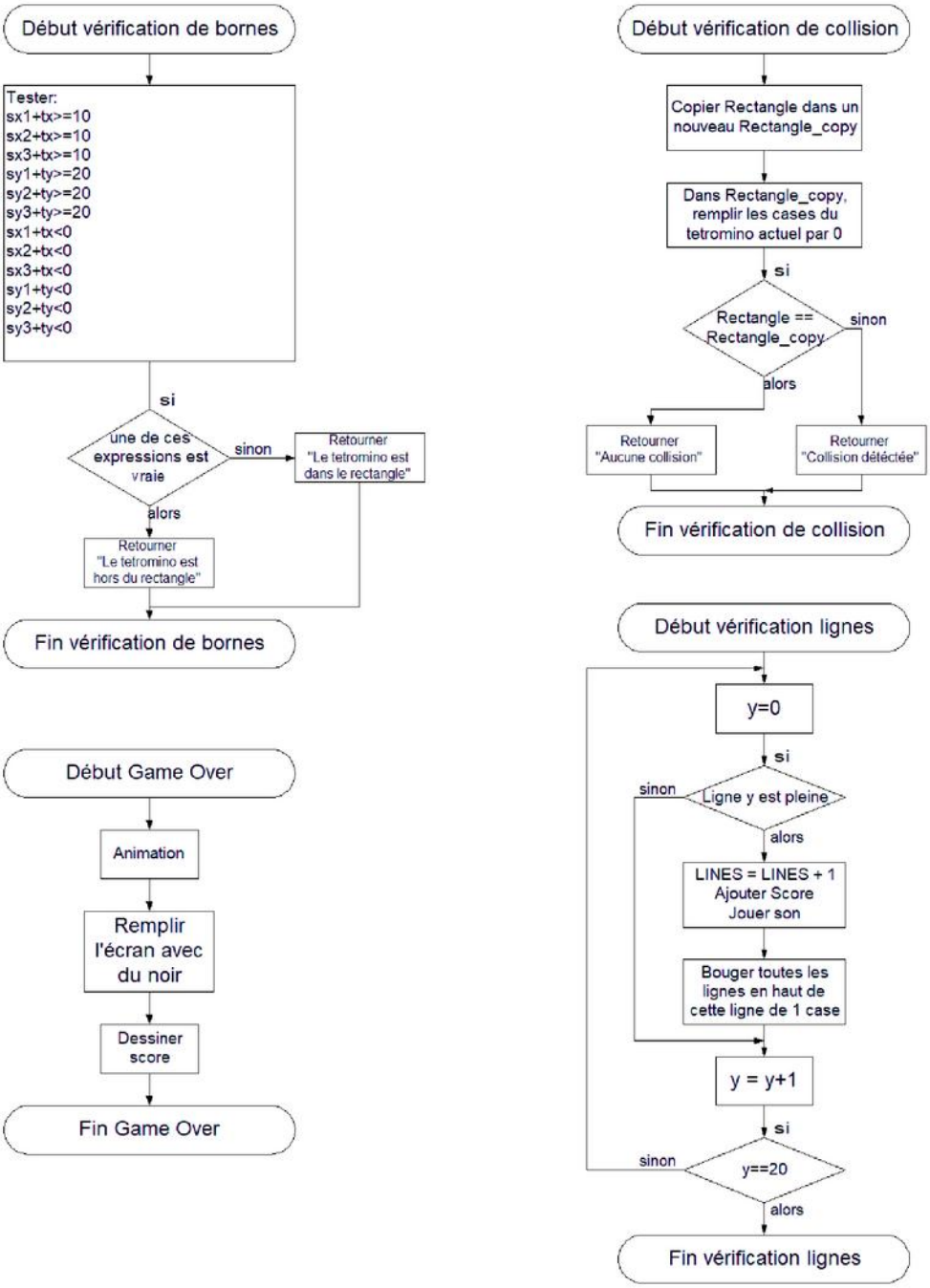


Figure 21 – *Algorigramme de Tetris écrit en assembleur (suite).*

6.3 Organisation de la mémoire



Figure 22 – *Photo de l'écran titre.*



(a) *La police de caractères IBM Bios.*

(b) *Notre police de caractères pour les chiffres.*

Pour écrire du texte sur l'écran (Figure 22), nous avons importé une police de caractères majuscules en mémoire. La dimension de chaque lettre est 8x8 pixels (Figure 23a). Pour écrire le score sur l'écran, nous avons créé une police de caractères de 16x18 composée uniquement de chiffres (Figure 23b).

7 Implémentation du son

7.1 Fonctionnement physique

Pour implémenter le son, nous avons décidé de munir le processeur d'une sortie PWM (expliqué dans le chapitre 3.4). La valeur de chaque échantillon du signal varie entre 0 et 255, c'est pourquoi nous avons un son sur 8-bits. Le processeur, à chaque coup d'horloge, incrémente un compteur allant aussi de 0 à 255. Si ce compteur est plus petit que la valeur du signal, la sortie est à 1, sinon elle est à 0. Cela crée un signal carré, qui est ensuite envoyé sur le haut-parleur pour produire un son.

Comme l'horloge oscille à 40MHz, notre fréquence maximale d'échantillonnage est de $\frac{40'000'000}{256} = 156.25kHz$ (notre système a besoin de 256 coups d'horloge pour émettre un signal carré, comme dit précédemment). En effet, notre système peut produire un son d'une fréquence maximale de 78.125kHz, par le théorème de Nyquist-Shannon. [45] Ceci n'est pas un problème, car la fréquence maximale audible par l'Homme est de 20kHz. [46]

45. Ayfer Ozgur. *Lectures 9-10 : Sampling Theorem.* Page consultée le 12 Juillet 2024. URL : https://web.stanford.edu/class/engr76/lectures/lecture9-10.pdf.

46. Jim Davis. *HEARING.* Page consultée le 12 Juillet 2024. URL : hhttps://iu.pressbooks.pub/humanphys/chapter/hearing/.

7.2 Comment marche un son synthétisé



Figure 24 – *Un graphique de volume d'un son dans Serum, un synthétiseur virtuel.*

Un son a généralement une attaque, un maintien (hold, en anglais) et une décroissance (release, en anglais) (Figure 24). En effet, l'attaque est le temps que le son prend pour atteindre son volume maximal et le maintien est le temps que le son reste à son volume maximal. Ensuite, le son décroit vers son volume minimal. Pour simplifier, nous n'avons pas implémenté ni l'attaque ni le maintien, ce qui veut dire que tout son commence à jouer instantanément et s'arrête au fur et à mesure.

7.3 Le calcul du son

Pour implémenter le son, nous envoyons une nouvelle valeur dans les sorties PWM du processeur tous les `SOUND_FREQ` coups d'horloge (comme expliqué plus haut). `SOUND_FREQ` est déterminé par la fréquence du son que nous souhaitons jouer. Si `SOUND_FREQ` est grand, alors le son est grave, et si n est petit, alors le son est aigu. La méthode qui envoie les valeurs dans les sorties PWM est appelée `play_sound` et est appelée tous les 256 coups d'horloge. Cette méthode, à chaque fois qu'elle est appelée, incrémente `SOUND_PROGRESS` (le temps de lecture qui augmente au fil du temps) de 1 pour savoir où elle en est dans le son. Elle incrémente aussi à chaque fois qu'elle envoie une valeur dans les sorties PWM la variable `SOUND_PROGRESS2`.

Cette méthode utilise les variables `SOUND_PROGRESS`, `SOUND_PROGRESS2`, `SOUND_DECAY`, `SOUND_TYPE`, `SOUND_FREQ`, ainsi que `SOUND_TONAL_SHIFT`.

Soit x la prochaine valeur à mettre dans les sorties PWM. D'abord, le son est calculé : si le son est de type «bruit», alors x est générée aléatoirement par la méthode `xorshift` (un son produit aléatoirement sonnera comme du bruit blanc, donc comme le son émis par une radio lorsqu'il n'y a rien de transmis). Si le son est de type «carré», alors $x = 255$.

Soit $y =$ `SOUND_PROGRESS` $>>$ `SOUND_DECAY`. Puis, x est décalé de y vers la droite ($x = x >> y$). En effet, plus `SOUND_DECAY` est grand, moins `SOUND_PROGRESS` aura un impact sur x et plus le son durera longtemps. Notons que si `SOUND_PROGRESS` est un nombre très grand, alors aucun son ne sera joué.

Ensuite, pour faire des effets de son comme le décalage tonal, nous avons ajouté une variable `SOUND_TONAL_SHIFT`. Cette variable est ajoutée à `SOUND_FREQ` à chaque fois qu'un nouveau x est calculé (ce délai est géré par `SOUND_PROGRESS2`). Cela permet de changer la fréquence du son en temps réel.

Enfin, x est envoyé dans les sorties PWM.

S'il n'y a pas de son à jouer et que le type de son est «carré», alors $x = 0$ est envoyé dans les sorties PWM, pour permettre de générer une onde carrée.

7.4 Jeu de son

Pour jouer un son, il suffit de remettre les variables `SOUND_PROGRESS` et `SOUND_PROGRESS2` à 0, et de changer la valeur de `SOUND_FREQ` à la fréquence du son que l'on veut jouer, `SOUND_DECAY` à la durée du son, et `SOUND_TYPE` à «bruit» ou «carré», ainsi que `SOUND_TONAL_SHIFT` à la valeur qu'on souhaite. Puis, la méthode `play_sound` étant déjà appelée en boucle durant les délais, le son sera joué.

8 Résultats obtenus

8.1 Rappel

Sur les photos ci-dessous (Figures 25a, 25b, 27a et 26a), nous pouvons voir notre ordinateur en train de faire tourner Tetris.

Nous avons aussi créé un boitier qui peut être observé sur la figure 26a.



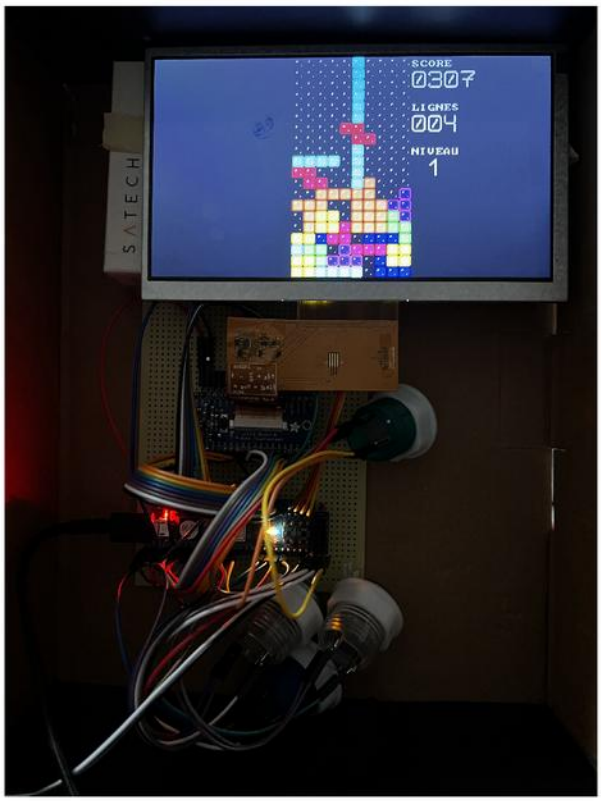
(a) *Le jeu Tetris en train de tourner sur notre processeur.*



(b) *Animation de fin de jeu.*



(a) *Fin de jeu.*



(a) *Une autre photo de notre processeur faisant tourner le jeu Tetris*

8.2 Détails techniques



Figure 28 – *Simulation du processeur faisant tourner Tetris, réagissant à l'appui d'un bouton.*

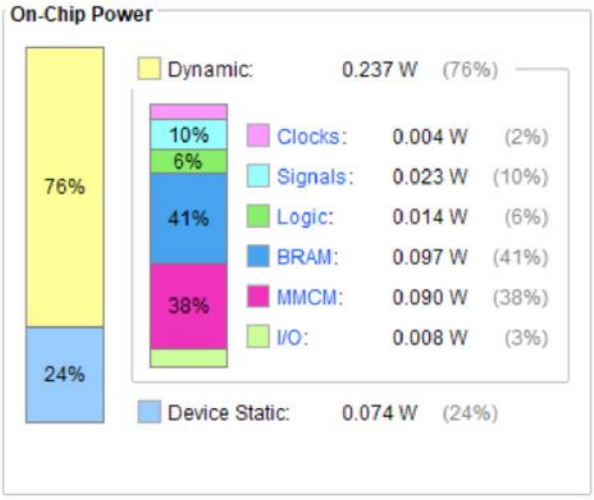


Figure 29 – *Consommation énergétique du processeur.*

Pour mesurer le temps écoulé entre l'appui d'un bouton et la réaction du processeur faisant tourner Tetris, nous l'avons simulé (Figure 28). Nous avons obtenu un temps de réaction du processeur de $26\mu s$ ($1016 - 990 = 26\mu s$, latence entre le point 1 et 2) après l'appui d'un bouton. Après avoir appuyé sur le bouton, le processeur calcule la nouvelle position du tetromino en $274\mu s$ ($1264 - 990 = 274\mu s$, latence entre le point 1 et 3), ce qui est très rapide.

Vivado nous a donné une estimation de la consommation énergétique du système, qui est de $0.237 + 0.074 = 0.311W$ (Figure 29).

8.3 Comparaison avec d'autres processeurs

Processeur	**Résolution [bits]**	**Vitesse d'horloge**	**MIPS (millions d'instructions par seconde)**	**MIPS/MHz**	**Registres multifonctionnels**
Notre processeur	64	$40MHz$	40	1	16 registres à 64 bits
STR71xF [47]	32	$50MHz$	45	0.9	31 registres à 32 bits
ESP32 [48]	32	$240MHz$	[49]142	0.6	4 registres à 16 bits

Processeur	**Mémoriser un nombre de 64bits [Coups d'horloge]**	**Additionner deux nombres de 64bits [Coups d'horloge]**	**Consommation énergétique générale [W]**	**Quantité maximale de RAM [octets]**	**Architecture**
Notre processeur	1	1	0.311	2^{63}	la notre
STR71xF	> 2	> 2	0.2	$64KB$	`ARM7TDMI` [50]
ESP32 [51]	4	> 4	$0.31 - 0.79$	$2^{32} = 4GB$	`RISC-V`

Table 1 – *Comparaison entre notre processeur et quelques processeurs actuels.*

Pour calculer le nombre de MIPS/MHz (millions d'instructions par seconde), nous avons simplement divisé le nombre d'instructions par seconde par la vitesse d'horloge. Notons que pour le processeur *ESP32*, nous avons utilisé un test de performance. [52] Pour calculer la consommation énergétique des processeurs,

52. NOPNOP2002. *esp-idf-benchmark*. Page consultée le 14 Juillet 2024. URL : https://github.com/nopnop2002/esp-idf-benchmark.

nous avons utilisé la formule $P = U \cdot I$, donc nous avons multiplié la tension et l'intensité pour mesurer la puissance. Pour le processeur *ESP32*, nous avons pris la consommation énergétique minimale et maximale lors de d'un fonctionnement normal ($P_{min} = U \cdot I_{min} = 3.3 \cdot 0.095 = 0.31W$, $P_{max} = U \cdot I_{max} = 3.3 \cdot 0.240 = 0.79W$ [53]). Puisque le bus de données de notre processeur est de 64 bits, et que le dernier bit est réservé pour la VRAM, nous avons calculé une quantité maximale de RAM de 2^{63} octets.

Pour calculer combien de coups d'horloge il faut pour mémoriser un nombre de 64 bits, nous nous sommes aidés de la documentation de chaque processeur. Pour le processeur *STR71xF*, il faut au moins 2 coups d'horloge pour mémoriser un nombre de 64 bits, puisqu'il faudrait mémoriser 2 registres de 32 bits. Pour le processeur *ESP32* dont les registres sont de 16 bits, il faudrait au moins 4 coups d'horloge pour mémoriser un nombre à 64 bits, puisqu'il faudrait mémoriser 4 registres. Pour notre processeur, il ne faut qu'un seul coup d'horloge pour mémoriser un nombre de 64 bits, puisque les registres sont de 64 bits.

Pour calculer combien de coups d'horloge il faut pour additionner deux nombres de 64 bits, nous avons utilisé la même méthode. Pour le processeur *STR71xF*, il faut au moins 2 coups d'horloge pour additionner deux nombres de 64 bits, puisqu'il faudrait additionner 32 bits par 32 bits et additionner les retenues. Pour le processeur *ESP32*, il faudrait au moins 4 coups d'horloge pour additionner deux nombres de 64 bits, puisqu'il faudrait additionner 16 bits par 16 bits et additionner les retenues. Pour notre processeur, il ne faut qu'un seul coup d'horloge pour additionner deux nombres de 64 bits, puisque la résolution des registres est de 64 bits.

8.4 Analyse des résultats

Comme nous pouvons le voir, notre processeur consomme plus d'énergie que le processeur *STR71xF*, mais moins que le processeur *ESP32*. Cependant, comme notre processeur est réalisé sur un FPGA, il a une vitesse d'horloge beaucoup plus basse que les autres processeurs. S'il était réalisé directement sur une puce, il pourrait avoir une vitesse d'horloge plus élevée, et donc une performance plus grande, puisque cela augmenterait aussi les instructions par seconde traitée par notre processeur. Notons que, comme celui-ci a une résolution de 64 bits, il prend plus de place sur le FPGA, et donc ne peut pas avoir une vitesse d'horloge aussi élevée que les autres processeurs dont la résolution est de 32 bits. Comme notre processeur est très simple, il ne s'arrête pas pour exécuter des instructions complexes, et donc il peut traiter plus d'instructions par coup d'horloge que les autres processeurs (MIPS/MHz). Enfin, notre processeur a une quantité maximale de RAM plus grande que les autres processeurs, puisque la résolution est de 64 bits.

53. Khaled Magdy. *ESP32 Sleep Modes & Power Consumption in Each Mode*. Page consultée le 14 Juillet 2024. URL : https://deepbluembedded.com/esp32-sleep-modes-power-consumption/.

9 Conclusion

9.1 Rappel

Ce projet nous a permis l'acquisition de beaucoup de nouvelles connaissances, notamment la conception d'architectures de microprocesseurs, les aspects différents d'implémentation d'éléments clés de celles-ci, et la création d'un système intégral étant capable de faire tourner divers logiciels, notamment ceux qui utilisent des périphériques externes comme une sortie vidéo ou un haut-parleur. Nous nous sommes aussi familiarisés avec la technologie FPGA, ainsi que les outils de simulation et de programmation de ceux-ci. Il était aussi nécessaire d'approfondir les connaissances liées aux langages de description de matériel (Verilog). Nous avons aussi développé des outils permettant de nous faciliter la traduction de l'assembleur dans lequel nous avons notamment écrit Tetris, afin de pouvoir ensuite le traduire en langage machine.

Point par point, après la création de l'architecture de notre microprocesseur, le modèle en Verilog a été développé, simulé et testé sur le FPGA. En plus, à l'aide de l'ajout de plusieurs modules à notre processeur, nous avons abouti à un système permettant de faire tourner des logiciels complexes et interactifs (notamment le jeu *Tetris*). Puis, la plateforme a été intégrée dans le FPGA à l'aide du logiciel *Vivado*. Ceci nous a permis d'avoir un ordinateur de jeu fonctionnel, performant et polyvalent. Pour programmer *Tetris* sur celui-ci, nous avons d'abord écrit le simulateur de ce jeu en C pour étudier la logique de ce jeu de plus près. Puis, nous avons traduit l'algorithme en l'assembleur de notre propre processeur tout en l'optimisant pour garantir la rapidité de son fonctionnement sur notre système. Notons aussi que ce fait a nécessité la création d'un compilateur d'assembleur (écrit en Python) qui traduit celui-ci en langage machine. Enfin, nous avons fait tourner Tetris sur notre système avec du son.

Nous sommes donc parvenus à la création d'un ordinateur complet, capable de permettre l'exécution de Tetris accompagné d'effets sonores, en ayant une consommation énergétique aussi basse que $0.311W$ et en ayant une fréquence d'horloge aussi lente que $40MHz$. Cela nous montre qu'en optimisant les programmes, on peut faire tourner des jeux même avec des processeurs lents en temps réel.

9.2 Critique

Nous avons d'abord implémenté le processeur. Puis, nous l'avons testé avec succès. Nous avons ensuite écrit un compilateur d'assembleur pour pouvoir implémenter Tetris. Après avoir programmé Tetris, nous avons muni le processeur de sorties PWM pour pouvoir jouer du son. Cette démarche s'est avérée correcte, puisqu'elle nous a permis de parvenir à notre but.

Lors de notre démarche, au moment d'ajouter des sorties GPIO, nous aurions pu prévoir l'ajout de sorties PWM pour le son, pour le faire au même moment.

Nous aurions pu implémenter les interruptions qui permettraient au processeur de ne pas scanner sans arrêt les périphériques, mais d'être averti lors d'un changement (par exemple, l'appui sur un bouton). Nous ne les avons pas implémentés, car nous étions limités par le temps. Nous aurions pu aussi implémenter des timers (qui sont présents dans presque tous les autres processeurs) qui permettraient à un système d'exploitation d'alterner les tâches, et de connaître l'heure qu'il est précisément.

En plus, la plupart des processeurs sont munis d'un signal nommé *WAIT*, qui arrête le processeur. Ce signal est utilisé dans des systèmes à plusieurs cœurs (plusieurs processeurs mis ensemble) par la mémoire pour dire aux cœurs du microprocesseur qu'elle n'est pas prête à être utilisée. Nous n'avons pas non plus implémenté un «arbitre» d'accès à la mémoire pour que celle-ci puisse être prise en charge par un processeur à plusieurs cœurs. Nous aurions pu implémenter le support pour avoir plusieurs cœurs, mais nous ne l'avons pas fait puisqu'il y a beaucoup de processeurs qui ne sont pas multicœurs.

L'implémentation d'une horloge externe aurait été bénéfique, pour cadencer le processeur et donc son taux d'échantillonnage du son, pour éviter l'oscillement de fréquence ou encore d'autres problèmes.

Nous aurions pu donner plus de registres à notre processeur, par exemple 32. De nos jours, certains processeurs (comme MIPS [54]) ont 32 registres. Cependant, nous sommes parvenus à faire tourner Tetris sur celui-ci avec 16. Les processeurs ayant une architecture x86 ne sont dotés que de 8 registres. [55]

Aussi, nous aurions pu ajouter des instructions de multiplication et de division. L'ajout de telles instructions nous aurait facilité la tâche lors de la multiplication par 10. Lors de l'élaboration de programmes plus compliqués, cela faciliterait aussi la tâche au programmeur. Cependant, l'ajout de telles instructions nécessiterait un ajout d'un pipeline, puisqu'une multiplication et une division auront besoin de plusieurs coups d'horloge pour le calcul du résultat. Nous ne l'avons pas implémenté pour ne pas implémenter de pipeline pour les calculs pour ne pas compliquer notre processeur davantage.

Nous aurions aussi pu ajouter les opérations sur les nombres à virgule flottante (nombres à virgule). Cela faciliterait davantage le calcul scientifique, mais cela nécessiterait l'ajout d'une unité à virgule flottante avec un pipeline (FPU, *floating point unit*, en anglais) qui s'occuperait du calcul des opérations

54. Chen, Novick et Shimano, *MIPS*, op. cit.
55. Microsoft, *x86 Architecture*, op. cit.

concernant ces nombres. En étant complexe, cette unité prendra beaucoup de place sur le FPGA et ralentira le processeur. Nous ne l'avons pas implémenté pour que notre processeur reste simple.

Si nous avions développé un compilateur C, le temps de programmation de Tetris aurait été plus petit. Cependant, la création d'un compilateur C nécessite beaucoup de temps et est très complexe, c'est pourquoi ce jeu a été complètement écrit en assembleur. Nous avons aussi fait ce choix pour pouvoir optimiser davantage le système. Par exemple, avec une implémentation du son écrite en C, il serait difficile de cadencer le jeu du son puisque C ne donne pas directement accès au nombre de coups d'horloge utilisés, que nous avions utilisé lors de notre programmation du jeu en assembleur pour cadencer les délais précisément.

9.3 Pour aller plus loin

Nous pourrions développer un système commercial sur une puce (c'est-à-dire un système complet embarqué sur un seul circuit intégré, en anglais *system on a chip* ou SoC) à partir de notre microprocesseur qui pourrait être la base d'un ordinateur portable ou d'une tablette à très basse consommation. Nous pourrions également concevoir un système d'exploitation complet, optimisé pour permettre l'utilisation d'un ordinateur portable basé sur notre microprocesseur, alimenté par des panneaux solaires. Si ce projet réussit, un ordinateur de ce type pourrait non seulement être un concurrent redoutable face à ceux disponibles sur le marché actuel, mais aussi devenir un outil éducatif précieux pour les populations les plus défavorisées.

10 Remerciements

Nous tenons à remercier notre directeur de Travail de Maturité, Thomas Morel.
Nous tenons aussi à remercier notre expert, René Beuchat, pour son aide précieuse.
Nous remercions également nos parents de nous avoir appris la programmation depuis notre plus jeune âge.

11 Bibliographie

Abdelrazek, Ahmed Fathy Mohammed. *Exception and Interrupt Handling in ARM*. Page consultée le 27 Mai 2024. URL : http://classweb.ece.umd.edu/enee447.S2016/ARM-Documentation/ARM-Interrupts-3.pdf.

Adafruit. *40-pin TFT Friend - FPC Breakout with LED Backlight Driver*. Page consultée le 15 Juillet 2024. URL : https://adafru.it/1932.

— *7.0" 40-pin TFT Display - 800x480 without Touchscreen*. Page consultée le 15 Juillet 2024. URL : https://adafru.it/2353.

— *Mini LED Arcade Button - 24mm Translucent Clear*. Page consultée le 15 Juillet 2024. URL : https://adafru.it/3429.

Advanced Micro Devices, Inc. *Adaptive SoCs and FPGAs*. Page consultée le 31 Mai 2024. URL : https://www.amd.com/en/products/adaptive-socs-and-fpgas.html.

— *AMD Artix™ 7 FPGAs*. Page consultée le 31 Mai 2024. URL : https://www.amd.com/en/products/adaptive-socs-and-fpgas/fpga/artix-7.html.

— *Vivado Overview*. Page consultée le 31 Mai 2024. URL : https://www.xilinx.com/products/design-tools/vivado.html.

Alan Bradley, PC MAG. *What Is VRAM? Why Games Are Using it More, and How to Optimize Your PC*. Page consultée le 3 Juin 2024. URL : https://www.pcmag.com/how-to/what-is-vram-why-games-are-using-it-more-and-how-to-optimize-your-pc.

Arduino S.r.l. *Arduino Uno Rev3*. Page consultée le 26 Mai 2024. URL : https://store.arduino.cc/products/arduino-uno-rev3.

ARM Limited. *ARM7TDMI-S Technical Reference Manual*. Page consultée le 13 Juillet 2024. URL : https://documentation-service.arm.com/static/5e8e13a9fd977155116a3368.

Arm Limited. *CPU Architecture*. Page consultée le 1er Juin 2024. URL : https://www.arm.com/architecture/cpu.

Atmel Corporation. *8-bit Atmel Microcontroller with 128KBytes In-System Programmable Flash*. Page consultée le 27 Mai 2024. URL : https://ww1.microchip.com/downloads/en/DeviceDoc/doc2467.pdf.

— *8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash*. Page consultée le 26 Mai 2024. URL : http://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf.

Brian Bailey - semiengineering.com. *What Will That Chip Cost?* Page consultée le 10 Juillet 2024. URL : https://semiengineering.com/what-will-that-chip-cost/.

Cassel, David. *Which Programming Languages Use the Least Electricity?* Page consultée le 27 Mars 2024. URL : https://thenewstack.io/which-programming-languages-use-the-least-electricity/.

Chen, Crystal, Greg Novick et Kirk Shimano. *How Pipelining Works*. Page consultée le 27 Mai 2024. URL : https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/pipelining/index.html.

— *MIPS*. Page consultée le 10 Juillet 2024. URL : https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/mips/index.html.

codecademy.com. *Compiling*. Page consultée le 23 Juin 2024. URL : https://www.codecademy.com/resources/docs/c/compiling.

Davis, Jim. *HEARING*. Page consultée le 12 Juillet 2024. URL : hhttps://iu.pressbooks.pub/humanphys/chapter/hearing/.

Diferr.com. *10 Differences Between verilog and vhdl*. Page consultée le 26 Mai 2024. URL : https://diferr.com/difference-between-verilog-and-vhdl/.

Digilent. *Cmod A7-35T : Breadboardable Artix-7 FPGA Module*. Page consultée le 31 Mai 2024. URL : https://digilent.com/shop/cmod-a7-35t-breadboardable-artix-7-fpga-module/.

— *Digilent – Start Smart, Build Brilliant*. Page consultée le 1er Juin 2024. URL : https://digilent.com/.

Espressif Systems (Shanghai) Co., Ltd. *ESP32 Technical Reference Manual*. Page consultée le 14 Juillet 2024. URL : https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.

Fog, Agner. *Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD, and VIA CPUs*. Page consultée le 27 Mai 2024. URL : https://www.agner.org/optimize/instruction_tables.pdf.

Intel Corporation. *Moore's Law - The past, present and future of Gordon Moore's golden rule for the semiconductor industry. [En Ligne]*. Page consultée le 17 Mars 2024. URL : https://www.intel.com/content/www/us/en/newsroom/resources/moores-law.html.

Jean-Michel Muller et al. *Handbook of Floating-Point Arithmetic*. Page consultée le 27 Mai 2024. URL : https://ieeemilestones.ethw.org/w/images/5/54/Handbook_Floating_Point_Chapter_3.pdf.

Khaled Magdy. *ESP32 Sleep Modes & Power Consumption in Each Mode*. Page consultée le 14 Juillet 2024. URL : https://deepbluembedded.com/esp32-sleep-modes-power-consumption/.

Khan, Muhammad Umair et al. *Measuring power consumption in mobile devices for energy sustainable app development : A comparative study and challenges*. Page consultée le 27 Mars 2024. URL : https://www.sciencedirect.com/science/article/abs/pii/S2210537921000780.

Lenovo. *ALU : Arithmetic Logic Unit Key Components & Logic Gates*. Page consultée le 7 Avril 2024. URL : https://www.lenovo.com/ph/en/glossary/arithmetic-logic-unit/.

libsdl-org. *Simple DirectMedia Layer (SDL) Version 3.0*. Page consultée le 23 Juin 2024. URL : https://github.com/libsdl-org/SDL.

Microsoft. *x86 Architecture*. Page consultée le 1er Juin 2024. URL : https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/x86-architecture.

Mouser Electronics, Inc. *GF0876 CUI Devices*. Page consultée le 15 Juillet 2024. URL : https://www.mouser.ch/ProductDetail/CUI-Devices/GF0876?qs=WyjlAZoYn5229XpWUmpA%252Bg%3D%3D.

NIDEC Group. *Calculating Infinity : The Paradox Of Moore's Law [En Ligne]*. Page consultée le 26 Mars 2024. URL : https://acim.nidec.com/drives/control-techniques/news-and-media/blog/insights/calculating-infinity-the-paradox-of-moores-law.

nopnop2002. *esp-idf-benchmark*. Page consultée le 14 Juillet 2024. URL : https://github.com/nopnop2002/esp-idf-benchmark.

Ozgur, Ayfer. *Lectures 9-10 : Sampling Theorem*. Page consultée le 12 Juillet 2024. URL : https://web.stanford.edu/class/engr76/lectures/lecture9-10.pdf.

PCWorld - IDG Communications, Inc. *Shopping for an older CPU ? Don't go past this—or you'll regret it*. Page consultée le 27 Mars 2024. URL : https://www.pcworld.com/article/539090/shopping-for-an-older-cpu-dont-go-past-this-or-youll-regret-it.html.

Pratt, Vaughan. *Anatomy of the Pentium Bug*. Page consultée le 10 Juillet 2024. URL : http://boole.stanford.edu/pub/anapent.pdf.

SAMSUNG. *One UI Beta Program - Experience the latest One UI with Android 14*. Page consultée le 27 Mars 2024. URL : https://developer.samsung.com/one-ui-beta.

— *Samsung Galaxy S24 Ultra*. Page consultée le 27 Mars 2024. URL : https://www.samsung.com/us/smartphones/galaxy-s24-ultra/.

Semenov Sergei - histrf.ru. *How a Soviet programmer made the worlds most popular game*. Page consultée le 23 Juin 2024. *Titre traduit*. URL : https://histrf.ru/read/articles/skaz-pro-to-kak-sovetskiy-programmist-samuyu-populyarnuyu-igru-v-mire-izobrel.

STMicroelectronics. *STR71xF*. Page consultée le 13 Juillet 2024. URL : https://media.digikey.com/pdf/Data%20Sheets/ST%20Microelectronics%20PDFS/STR71xF.pdf.

Tennberg, Patrik. *The lost art of code optimization*. Page consultée le 27 Mars 2024. URL : https://www.linkedin.com/pulse/lost-art-optimization-patrik-tennberg.

The Conversation Media Group Ltd, Arnab Hazari. *The future of electronics is light*. Page consultée le 27 Mars 2024. URL : https://theconversation.com/the-future-of-electronics-is-light-68903.

Volk, Andrew M., Peter A. Stoll et Paul Metrovich. *Recollections of Early Chip Development at Intel*. Page consultée le 26 Mai 2024. URL : https://deramp.com/downloads/mfe_archive/050-Component%20Specifications/Intel/Recollections%20of%20Early%20Chip%20Dev.pdf.

www.makeuseof.com, Ayush Jalan. *Why Do So Many Developers Launch Their Apps on iOS Before Android ?* Page consultée le 27 Mars 2024. URL : https://www.makeuseof.com/why-are-cross-platform-apps-released-on-ios-first/.

www.makeuseof.com, Joel Lee. *30 Years Of Tetris : A History Of The World's Most Beloved Puzzle Game*. Page consultée le 27 Mars 2024. URL : https://www.makeuseof.com/tag/30-years-tetris-history-worlds-beloved-puzzle-game/.

Zilog, Inc. *Z80 CPU*. Page consultée le 26 Mai 2024. URL : https://www.zilog.com/docs/z80/um0080.pdf.

Gymnase de Nyon

Travail de Maturité 2024

Comment créer et optimiser un système à base d'un microprocesseur pour un jeu vidéo comme Tetris

ADRIAN VLASOV

Classe : 3M02

Annexes

Pages 33-159

Direction : M. Thomas Morel

Expert : M. René Beuchat

Février 2024

12 Annexes

12.1 Détails de l'implémentation

Pour que les instructions soient écrites dans la RAM lors du démarrage du processeur pour après être lues à partir de la RAM, nous nous sommes aidés du mot-clé `initial` et de la commande `$readmemh(..., <registre>)` de Verilog. Pour accéder à la VRAM, par exemple, colorier un pixel de l'écran en blanc, nous avons choisi que le 64ème bit de l'adresse de la mémoire définirait le multiplexage entre la RAM et la VRAM. Si le 64ème bit de l'adresse est fixé à 1, alors cela désigne la VRAM, sinon la RAM. Nous avons aussi implémenté l'accès aux périphériques (donc *GPIO*, qui est l'acronyme de *General-Purpose Input-Output*, dont on n'a que 16 de libres, car l'écran est connecté). Lorsque le 10ème bit de l'adresse de mémoire est à 1, alors le premier bit correspond au bouton A, le deuxième bit correspond au bouton B et le troisième bit correspond à la LED A du circuit. Les 5 bits restants sont mis à 0. L'octet suivant, donc le deuxième, correspond aux 8 broches de *GPIO* entrantes. L'octet numéro 3 correspond aux 8 broches de *GPIO* sortantes. Les 40 derniers bits sont mis à 0 sinon. Les boutons A et B, ainsi que la LED A, correspond aux boutons et leds déjà soudés sur le circuit Cmod A7.

12.2 Problèmes rencontrés

Si l'adresse de l'insrtuction, *instr_addr*, est trop grand, comme les instructions et les données sont contenues dans la même mémoire, le processeur lira des données comme si elles étaient des instructions, ce qui aura pour conséquence un comportement inattendu.

12.3 Format du jeu d'instructions

Nous allons expliciter notre jeu d'instructions. Mais d'abord, pour lire le jeu d'instructions, le format est le suivant :

Rd signifie l'adresse sur 4 bits du registre destination, que nous modifions

Rs désigne l'adresse sur 4 bits d'un registre source, que nous souhaitons lire

Md indique l'adresse sur 4 bits du registre qui contient l'adresse destination sur 64 bits de la mémoire externe, donc l'adresse à laquelle nous voulons écrire dans la mémoire

Ms signifie l'adresse sur 4 bits du registre qui contient l'adresse de la source 64 bits de la mémoire externe, donc l'adresse à laquelle nous voulons lire dans la mémoire

***Id*$_{xxx}$** indique des données immédiates, donc des données directement présentes dans l'instruction, à xxx bits. Nous allons parfois évoquer cette notion en tant que *valeur littérale*.

***Id*$_{hw}$** indique des données immédiates à 32 bits. ≪hw≫ signifie *half word*, qui indique la moitié d'un mot de processeur. Dans notre cas, un mot de processeur correspond à 64 bits, c'est pourquoi un ≪hw≫ a une longueur de 32 bits. Nous allons parfois évoquer cette notion en tant que *valeur littérale*.

s a une longueur de 8 bits, et montre à l'unité de mémoire les octets à modifier. Un 1 à la position x montre à l'unité de mémoire de toucher l'octet numéro x, tandis qu'un 0 dit de ne rien faire. Par exemple, un s de 00000011 (Figure 30) dira à l'unité de mémoire de ne remplacer que les deux premiers bytes des données à une adresse donnée.

7	6	5	4	3	2	1	0
0	0	0	0	0	0	1	1
Byte 7	Byte 6	Byte 5	Byte 4	Byte 3	Byte 2	Byte 1	Byte 0

Figure 30 – *Représentation de l'exemple* $s = 00000011$. *Les bytes non touchés sont coloriés en gris.*

c désigne un seul bit de contrôle dont l'utilité sera spécifiée dans le jeu d'instructions ci-dessous

f qui a une longueur de 4 bits (Figure 31) est un *flag mask*, un masque d'indicateurs, qui montre quels indicateurs devraient être modifiés par l'ALU au moment d'exécution de l'instruction. Le premier bit (compté sous forme de 0001) indique la modification de l'indicateur *Overflow*. Le deuxième bit (0010) indique l'autorisation de la modification de l'indicateur *Carry*. Le troisième bit autorise l'ALU à changer l'indicateur *Negative* et enfin le quatrième bit laisse l'ALU modifier l'indicateur *Zero*.

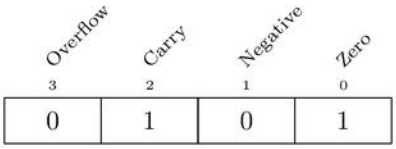


Figure 31 – *Exemple d'un $f = 0101$*

$f_{x\ Z,N,C,O}$ est similaire à *f*, sauf qu'il n'y a que les x derniers bits des 4 bits qui indiqueront un changement. Par exemple, comme nos indicateurs sont ordonnés de la manière Z, N, C, O, un $f_{2\ Z,N}$ désignera 1100.

fc désigne la combinaison d'indicateurs souhaitée pour quelques contrôles. Pour ce faire, voici la liste de valeurs qu'il peut prendre et ses explications :

- **0 ou *000*** signifie *pas égal*, ou quand l'indicateur $Z = 0$
- **1 ou *001*** désigne *égal*, ou quand l'indicateur $Z = 1$. Ceci est vrai, car, pour comparer des valeurs, nous faisons une soustraction. Lorsqu'elles sont égales, l'indicateur *Zero* est enclenché.
- **2 ou *010*** désigne *plus grand ou égal*, ou quand l'indicateur $C = 0$. Ceci est vrai, car lors d'une soustraction de deux nombres, si le premier était plus grand que le deuxième, il n'y aurait pas de débordement.
- **3 ou *011*** signifie *strictement plus petit*, ou quand l'indicateur $C = 1$.
- **4 ou *100*** désigne *strictement plus grand*, ou quand $Z = C = 0$. Même explication que pour *plus grand ou égal*, sauf que nous excluons l'égalité, donc c'est pourquoi $Z = 0$.
- **5 ou *101*** signifie *plus petit ou égal*, ou quand $C = 1$ ou $Z = 1$.
- **6 ou *110*** désigne *positif*, ou quand $N = 0$, donc quand le nombre n'est pas négatif.
- **7 ou *111*** désigne *négatif*, ou quand $N = 1$, donc quand le résultat est négatif.

12.4 Champs de bits

Les instructions du processeur sont codées sous forme binaire. C'est pourquoi, pour illustrer chaque format d'instruction, nous avons représenté ci-dessous les différents formats possibles de champ de bits.

1. **Instruction arithmétique normale** Une instruction de ce type serait , par exemple, *addition* ou *soustraction*. L'exemple (en dessous du champ de bits) consiste en l'instruction *add* redéfinissant $R_5 := R_5 + R_0$ et en modifiant tous les indicateurs.

63 – 58	54	50	46	42	40	0
Opération	Rd	Rs_1	Rs_2	f	c	Non utilisé
000000	0101	0101	0000	1111	0	$000\cdots000$

Figure 32 – *Champ de bits d'une instruction arithmétique normale avec un exemple en dessous.*

2. **Instruction arithmétique avec valeur littérale** Des instruction de ce type sont, par exemple, *addi* et *subi*. L'exemple (sous le champ de bits) consiste en l'instruction *addi* ajoutant $1'234'567'890$ à R_0 et, en faisant cela, en modifiant tous les indicateurs.

63 – 58	54	50	18	14	0
Opération	Rd	Rs	Id_{hw}	f	Non utilisé
000001	0000	0000	0100 1001 1001 0110 0010 0101 1111 1010	1111	$000\cdots000$
			1'234'567'890 en base 10		

Figure 33 – *Champ de bits d'une instruction arithmétique avec valeur littérale suivie d'un exemple.*

3. **Instruction logique normale** Par exemple, le ET logique et le OU logique utilisent ce format. Les deux derniers bits de $f_{2\ Z,N}$ sont inmodifiables. L'exemple consiste en l'instruction *and* redéfinissant $R_5 := R_5|R_0$ et en ne modifiant que les indicateurs Z et N.

63 – 58	54	50	46	42		0
Opération	Rd	Rs_1	Rs_2	f	00	Non utilisé
000101	0001	0001	0000	11	00	$000\cdots000$

Figure 34 – *Champ de bits d'une instruction logique normale avec un exemple en dessous.*

4. **Instruction logique avec valeur littérale** Par exemple, *andi* et *ori* utilisent ce format. L'exemple consiste en l'instruction *andi* faisant l'opération ET entre $1'234'567'890$ et R_0 et mettant le résultat dans R_0. Cette instruction ne modifie que les indicateurs Z et N.

63 – 58	54	50 – 18		15			0
Opération	Rd	Rs	Id_{hw}	c	f	00	Non utilisé
000101	0000	0000	0100 1001 1001 0110 0010 0101 1111 1010	0	11	00	$000\cdots000$

1'234'567'890 en base 10

Figure 35 – *Champ de bits d'une instruction logique avec valeur littérale suivie d'un exemple.*

5. **Instruction à un seul registre modifiant 2 indicateurs** Par exemple, *neg* et *rrc* sont ce de type. L'exemple consiste en l'instruction *neg* qui fait le complément à deux de R_0. Elle modifie les indicateurs Z et N.

63 – 58	54	50		0
Opération	Rd	f	00	Non utilisé
001010	0000	11	00	$000\cdots000$

Figure 36 – *Champ de bits d'une instruction à un seul registre modifiant seulement 2 indicateurs suivie d'un exemple.*

6. **Instruction à un seul registre modifiant 3 indicateurs** Par exemple, *asr* est de ce type. L'exemple, consiste en l'instruction *asr* qui décale R_0 vers la droite en laissant le 64ème en place. Elle modifie les indicateurs Z et N, mais pas C en raison du 0 à la première position de $f_{3\ Z,N,C}$.

63 – 58	54	51		0
Opération	Rd	f	0	Non utilisé
001010	0000	110	0	$000\cdots000$

Figure 37 – *Champ de bits d'une instruction à un seul registre modifiant seulement 3 indicateurs suivie d'un exemple.*

7. **Instruction de décalage logique modifiant 3 indicateurs** Par exemple, *lsr* est de ce type. L'exemple, consiste en l'instruction *lsr* qui décale R_0 de $R_2 + 3$ vers la droite. Cette instruction, représentée en exemple, ne modifie aucun indicateur, car nous avons réglé f à 000.

63 – 58	54	50	46 – 14		11		0
Opération	Rd	Rs_1	Rs_2	Id_{hw}	f	0	Non utilisé
001011	0000	0000	0010	0000 0000 0000 0000 0000 0000 0000 0011	000	0	$000\cdots000$

3 en base 10

Figure 38 – *Champ de bits d'une instruction de décalage logique modifiant seulement 3 indicateurs suivie d'un exemple.*

8. Instruction de décalage logique avec valeur littérale modifiant 3 indicateurs Par exemple, *lsri* est de ce type. L'exemple, consiste en l'instruction *lsri* qui décale R_0 de 3 vers la droite. Cette instruction, représentée en exemple, ne modifie aucun indicateur, car nous avons réglé f à 000.

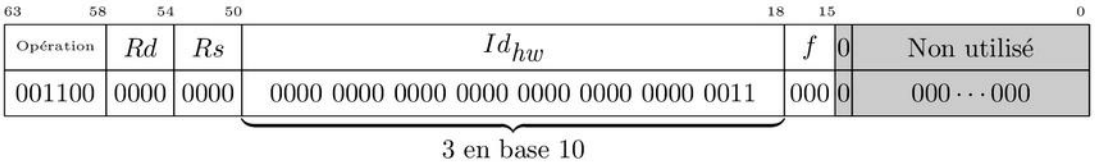


Figure 39 – *Champ de bits d'une instruction de décalage logique avec valeur littérale modifiant seulement 3 indicateurs suivie d'un exemple.*

9. Instruction à deux registres Par exemple, l'instruction *cp* est de ce type. Pour illustrer, nous avons ajouté un exemple consistant en l'instruction *cp* comparant R_1 et R_0.

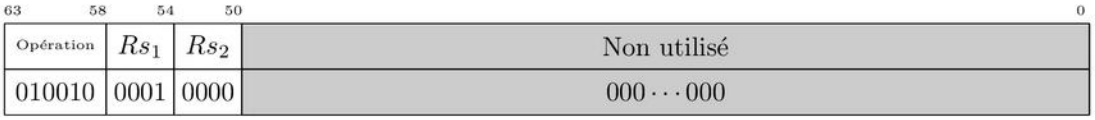


Figure 40 – *Champ de bits d'une instruction à deux registres suivie d'un exemple.*

10. Instruction à un registre et à une valeur littérale Par exemple, *cpi* utilise ce champ de bits. Pour illustrer, nous avons ajouté un exemple consistant en l'instruction *cpi* comparant R_0 et 42.

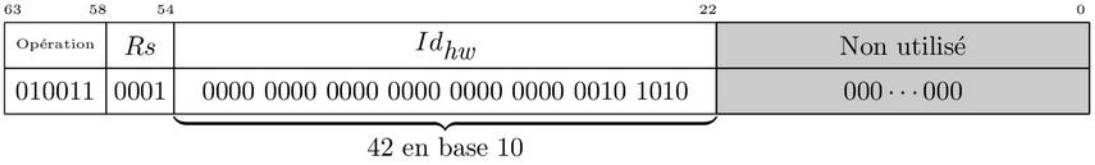


Figure 41 – *Champ de bits d'une instruction à un registre et à une valeur littérale suivie d'un exemple.*

11. Instruction à indicateurs seulement Par exemple, *fst* est de ce type. Pour illustrer, nous avons ajouté un exemple consistant en l'instruction *fst* qui, en étant exécutée, mettre les indicateurs Z et C à 1.

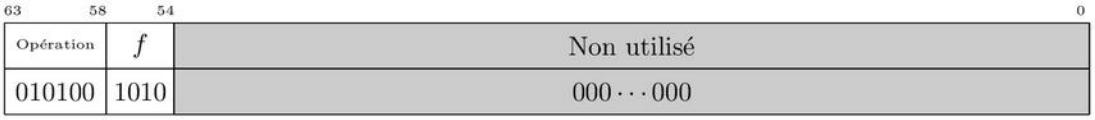


Figure 42 – *Champ de bits d'une instruction à indicateurs seulement suivie d'un exemple.*

12. Instruction à une valeur littérale Par exemple, *jmpi* est une instruction de ce type. L'exemple consiste en l'instruction *jmpi* qui oblige le processeur à sauter à l'adresse 42.

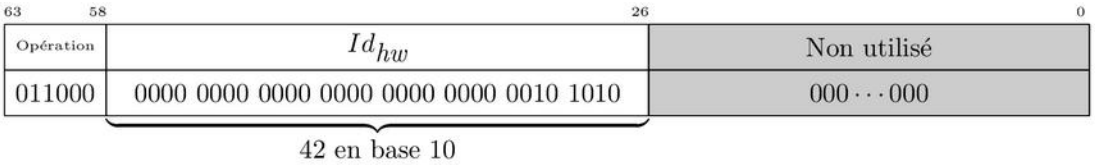


Figure 43 – *Champ de bits d'une instruction à une valeur littérale suivie d'un exemple.*

13. Instruction conditionelle de branchement à un registre et à une valeur littérale de 29 bits Par exemple, *bif* est de ce type. L'exemple consiste en l'instruction *bif* qui oblige le processeur à sauter à l'adresse $R_0 + 42$ si une comparaison exécutée avant (comme *cp*, par exemple) aurait enclenché les indicateurs C ou Z, donc si, pour les arguments de cette comparaison, la relation était *plus petit ou égal*.

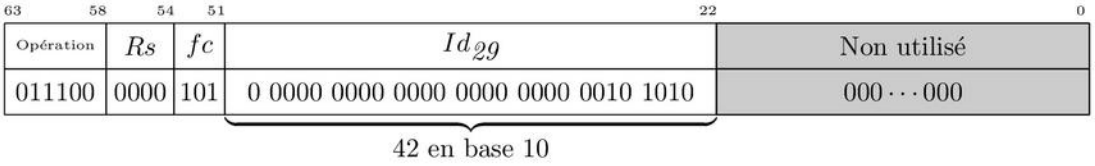


Figure 44 – *Champ de bits d'une instruction conditionelle de branchement à un registre et à une valeur littérale de 29 bits suivie d'un exemple.*

14. Instruction conditionelle de branchement à une valeur littérale de 29 bits Par exemple, *bifr* est de ce type. L'exemple consiste en l'instruction *bifi* qui oblige le processeur à sauter à l'adresse 42 si une comparaison exécutée avant (comme *cp*, par exemple) aurait mis l'indicateur C à 0, pour les arguments de cette comparaison, la relation était *plus grand ou égal*.

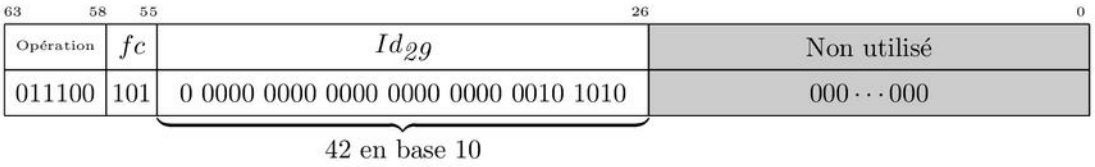


Figure 45 – *Champ de bits d'une instruction conditionelle de branchement à une valeur littérale de 29 bits suivie d'un exemple.*

15. Instruction à deux registres, une valeur littérale et à un mask de 8 bits Par exemple, *mov* est de ce type. L'exemple consiste en l'instruction *mem* qui met à l'adresse contenue dans R_0 les données contenues dans R_1, où nous avons décidé de n'inscrire que les bytes impairs (1, 3, 5 et 7).

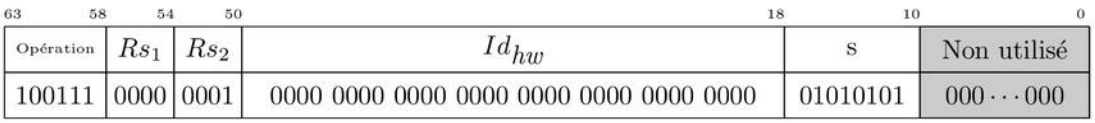


Figure 46 – *Champ de bits d'une instruction à deux registres, une valeur littérale et à un mask de 8 bits suivie d'un exemple.*

16. Instruction à un registre, une valeur littérale et à un mask de 8 bits Par exemple, *memi* utilise ce champ de bits. L'exemple consiste en l'instruction *memi* qui met à l'adresse contenue dans R_0 le nombre 42, où nous avons décidé d'inscrire tous les bits.

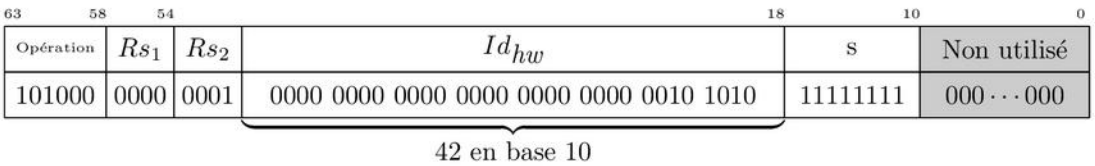


Figure 47 – *Champ de bits d'une instruction à un registre, une valeur littérale et à un mask de 8 bits suivie d'un exemple.*

17. Instruction à un registre, une valeur littérale et à un bit de contrôle Par exemple, *ldi* utilise ce champ de bits. L'exemple consiste en l'instruction *ldi* qui initialise les 32 premiers bits de R_0 au nombre 42.

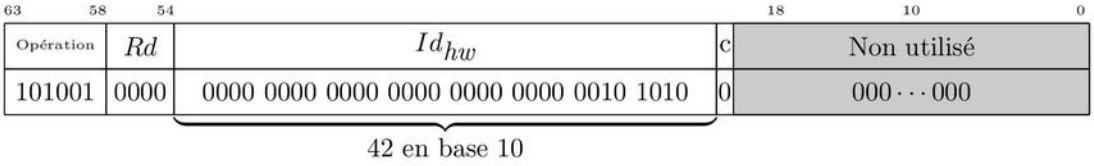


Figure 48 – *Champ de bits d'une instruction à un registre, une valeur littérale et à un bit de contrôle suivie d'un exemple.*

18. Instruction à un registre Par exemple, *sas* est de ce type. L'exemple consiste en l'instruction *sas* qui sauvegarde l'adresse d'une sous-routine dans le registre R_0.

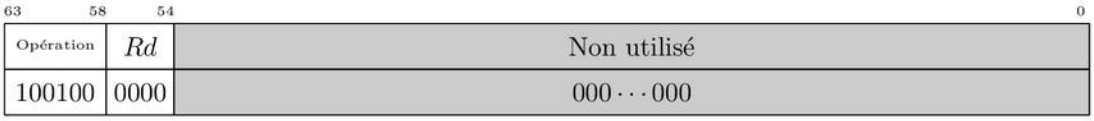


Figure 49 – *Champ de bits d'une instruction à un registre suivie d'un exemple.*

19. Instruction *nop* Le champ de bits pour l'instruction *nop* figure ci-dessous (Figure 50). Cette instruction ne fait strictement rien, c'est pourquoi elle n'a pas d'opérandes. Les numéreaux 37 et plus peuvent être utilisés pour représenter *nop*.

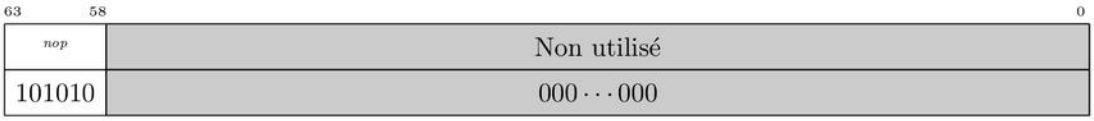


Figure 50 – *Champ de bits de l'instruction nop suivie d'un exemple.*

12.5 Jeu d'instructions

Voici le jeu d'instructions de notre processeur, dont chaque numéro correspondra au nombre d'instructions réel et dont le type de champ de bits sera mentionné à côté :

Format : Instruction	Champ de bits
0. *add* Rd, Rs_1, Rs_2, f, c	1
1. *addi* Rd, Rs, Id_{hw}, f	2
2. *sub* Rd, Rs_1, Rs_2, f, c	1
3. *subi* Rd, Rs, Id_{hw}, f	2
4. *and* Rd, Rs_1, Rs_2, $f_{2\ Z,N}$	3
5. *andi* Rd, Rs, Id_{hw}, c, $f_{2\ Z,N}$	4
6. *or* Rd, Rs_1, Rs_2, $f_{2\ Z,N}$	3
7. *ori* Rd, Rs, Id_{hw}, c, $f_{2\ Z,N}$	4
8. *xor* Rd, Rs_1, Rs_2, $f_{2\ Z,N}$	3
9. *xori* Rd, Rs, Id_{hw}, c, $f_{2\ Z,N}$	4
10. *neg* Rd, $f_{2\ Z,N}$	5
11. *lsr* Rd, Rs_1, Rs_2, Id_{hw}, $f_{3\ Z,N,C}$	7

12. ***lsri*** Rd, Rs, Id_{hw}, $f_{3\ Z,N,C}$ 8
13. ***lsl*** Rd, Rs_1, Rs_2, Id_{hw}, $f_{3\ Z,N,C}$ 7
14. ***lsli*** Rd, Rs, Id_{hw}, $f_{3\ Z,N,C}$ 8
15. ***rrc*** Rd, $f_{2\ Z,N}$ 5
16. ***rlc*** Rd, $f_{2\ Z,N}$ 5
17. ***asr*** Rd, $f_{3\ Z,N,C}$ 6
18. ***cp*** Rs_1, Rs_2 9
19. ***cpi*** Rd, Id_{hw} 10
20. ***fst*** f 11
21. ***fcl*** f 11
22. ***jmp*** Rs, Id_{hw} 10
23. ***jmpr*** Id_{hw} 12
24. ***jmpi*** Id_{hw} 12
25. ***cs*** Rs, Id_{hw} 10
26. ***csr*** Id_{hw} 12
27. ***csi*** Id_{hw} 12
28. ***bif*** Rs, fc, Id_{29} 13
29. ***bifr*** fc, Id_{29} 14
30. ***bifi*** fc, Id_{29} 14
31. ***bio*** Rs, Id_{hw} 10
32. ***bior*** Id_{hw} 12
33. ***bioi*** Id_{hw} 12
34. ***ret*** Id_{hw} 12
35. ***rti*** devait être la commande de retour d'interruption, nous avons décidé de nous en passer
36. ***sas*** Rd 18
37. ***sal*** Rs 18
38. ***mov*** Rd, Ms, Id_{hw}, s 15
39. ***mem*** Md, Rs, Id_{hw}, s 15
40. ***memi*** Md, Id_{hw}, s 16
41. ***ldi*** Rd, Id_{hw}, c 17
42 ou plus. ***nop*** 19

Voici les explications pour chaque instruction :

0 : ***add*** Rd**,** Rs_1**,** Rs_2**, f, c** additionne Rs_1 et Rs_2 ensemble et met le résultat dans Rd. Le programmeur pourra choisir quels indicateurs il faudrait autoriser avec *f*. Le bit de contrôle *c*, lorsqu'il est enclenché, utilisera le débordement d'avant, s'il y en a un, pour faire l'addition. Ceci est utile dans le cas d'addition de nombres ayant besoin de plus de place pour être stockés (par exemple, 128 bits). Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

1 : ***addi*** Rd**,** Rs**,** $Id_{\boldsymbol{hw}}$**, f** additionne Rs et le nombre qu'il y a dans Id_{hw} ensemble et met le résultat dans Rd. Le programmeur pourra aussi choisir quels indicateurs il faudrait autoriser avec *f*. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

2 : ***sub*** Rd**,** Rs_1**,** Rs_2**, f, c** soustrait Rs_2 à Rs_1 ensemble et met le résultat dans Rd. Le programmeur pourra choisir quels indicateurs il faudrait autoriser avec *f*. Le bit de contrôle *c*, lorsqu'il est enclenché, utilisera le débordement d'avant, s'il y en a un, pour faire la soustraction. Ceci est utile dans le cas de soustractions de nombres ayant besoin plus de place pour être stockés (par exemple, 128 bits). Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

3 : ***subi*** Rd**,** Rs**,** $Id_{\boldsymbol{hw}}$**, f** soustrait Id_{hw} à Rs et met le résultat dans Rd. Le programmeur pourra aussi choisir les indicateurs à autoriser avec *f*. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

4 : *and* Rd, Rs_1, Rs_2, $f_{2\ Z,N}$ Cette commande fait l'opération *OU* entre Rs_1 et Rs_2 et met le résultat dans Rd. Cette instruction ne génère que deux indicateurs, *Zero* et *Negative*, car, pour générer *Carry* ou *Overflow*, il faudrait faire une opération mathématique du type d'addition. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

5 : *andi* Rd, Rs, $Id_{\boldsymbol{hw}}$, **c**, $f_{2\ Z,N}$ Comme il n'y a que 32 bits dans Id_{hw}, en utilisant c, le programmeur pourrait choisir de faire l'opération ET logique sur les 32 premiers bits ou les 32 derniers bits du registre. Quand c est enclenché à 1, le processeur fait cette opération sur les 32 derniers bits du registre Rd, sinon sur les 32 premiers. Cette commande ne génère que deux indicateurs, *Zero* et *Negative*, pour la même raison que *and*.

6 : *or* Rd, Rs_1, Rs_2, $f_{2\ Z,N}$ fait l'opération *OU* entre Rs_1 et Rs_2 et met le résultat dans Rd. Cette commande ne génère que deux indicateurs, *Zero* et *Negative*, pour la même raison que *and*. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

7 : *ori* Rd, Rs, $Id_{\boldsymbol{hw}}$, **c**, $f_{2\ Z,N}$ Lorsque c est enclenché à 1, le processeur fait l'opération *OU* sur les 32 derniers bits entre le registre Rs et Id_{hw}, sinon sur les 32 premiers, et le met dans le registre à l'adresse Rd. Cette instruction ne génère que deux indicateurs, *Zero* et *Negative*.

8 : *xor* Rd, Rs_1, Rs_2, $f_{2\ Z,N}$ fait l'opération *OU exclusif*, appelée aussi *XOR*, entre Rs_1 et Rs_2 et met le résultat dans Rd. Cette commande ne génère que deux indicateurs, *Zero* et *Negative*. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

9 : *xori* Rd, Rs, $Id_{\boldsymbol{hw}}$, **c**, $f_{2\ Z,N}$ Lorsque c est enclenché à 1, le processeur fait l'opération *XOR*, ou *OU exclusif* sur les 32 derniers bits entre le registre Rs et Id_{hw}, sinon sur les 32 premiers, et le met dans le registre à l'adresse Rd. Cette commande ne génère que deux indicateurs, *Zero* et *Negative*. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

10 : *neg* Rd, $f_{2\ Z,N}$ fait le complément à deux du nombre du registre Rd. Cette commande ne génère que deux indicateurs, *Zero* et *Negative*, pour la même raison que *and*. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

11 : *lsr* Rd, Rs_1, Rs_2, $Id_{\boldsymbol{hw}}$, $f_{3\ Z,N,C}$ décale le nombre de Rs_1 à droite Rs_2+Id_{hw}-fois puis l'inscrit dans Rd. Cette instruction génère *Zero* et *Negative*, comme toutes celles que nous avons listées jusqu'à maintenant et pour la même raison. *Carry* est généré aussi, mais cette fois-ci l'indicateur est mis au bit numéro $Rs_2 + Id_{hw} - 1$ de Rs_1, pour pouvoir décaler des nombres plus grands que 64 bits. Si $Rs_2 + Id_{hw} = 0$, alors *Carry* est mis à 0. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

12 : *lsri* Rd, Rs, $Id_{\boldsymbol{hw}}$, $f_{3\ Z,N,C}$ décale le nombre de Rs à droite Id_{hw}-fois puis l'inscrit dans Rd. Cette instruction génère *Zero* et *Negative*. *Carry* est généré aussi, mais cette fois-ci l'indicateur est mis au bit numéro $Id_{hw} - 1$ de Rs, pour la même raison que dans *lsr*. Si $Id_{hw} = 0$, alors *Carry* est mis à 0. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

13 : *lsl* Rd, Rs_1, Rs_2, $Id_{\boldsymbol{hw}}$, $f_{3\ Z,N,C}$ décale le nombre de Rs_1 à gauche $Rs_2 + Id_{hw}$-fois puis l'inscrit dans Rd. Cette instruction génère *Zero* et *Negative*. *Carry* est généré aussi et est fixé au bit numéro $64 - Rs_2 - Id_{hw}$ de Rs_1, pour pouvoir décaler des nombres plus grands que 64 bits. Si $Rs_2 + Id_{hw} = 0$, alors *Carry* est mis à 0. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

14 : *lsli* Rd, Rs, $Id_{\boldsymbol{hw}}$, $f_{3\ Z,N,C}$ décale le nombre de Rs à gauche Id_{hw}-fois puis l'inscrit dans Rd. Cette instruction génère *Zero* et *Negative*. *Carry* est généré et prend la valeur du bit numéro $64 - Id_{hw}$ de Rs, pour la même raison que dans *lsl*. Si $Id_{hw} = 0$, alors *Carry* est mis à 0. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

15 : *rrc* Rd, $f_{2\ Z,N}$ décale Rd de 1 vers la droite et met le 64ème bit à celui qui se trouvait dans l'indicateur *Carry* à cet instant précis. Cette instruction génère aussi *Zero* et *Negative*. Le nouveau *Carry* prend une valeur du bit numéro 0 de l'ancien Rd. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

16 : *rlc* Rd, $f_{2\ Z,N}$ décale Rd de 1 vers la gauche et met le premier bit à celui qui se trouvait dans l'indicateur *Carry* à cet instant précis. Cette instruction génère aussi *Zero* et *Negative*. Le nouveau *Carry* prend une valeur du 64ème bit de l'ancien Rd. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

17 : *asr* Rd**,** $f_{3\ Z,N,C}$ décale Rd de 1 vers la droite, mais laisse le 64ème bit en place. Ainsi le 64ème bit reste à la position 64, mais est aussi dupliqué à la position 63. Cette instruction génère *Zero* et *Negative*. *Carry* prend la valeur du tout premier bit de Rd avant l'opération. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

18 : *cp* Rs_1**,** Rs_2 soustrait Rs_2 à Rs_1, mais ne stocke pas le résultat. Elle ne stocke que les indicateurs *Zéro*, *Négatif* et *Carry*. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

19 : *cpi* Rd**,** $Id_{\boldsymbol{hw}}$ soustrait Id_{hw} à Rs_1, mais ne stocke pas le résultat. Elle ne stocke que les indicateurs *Zéro*, *Négatif* et *Carry*. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

20 : *fst* **f** enclenche les indicateurs basés sur f. Par exemple, un f valant 0101 enclencherait les indicateurs *Overflow* et *Négatif*. Les indicateurs seront immédiatement accessibles au prochain coup d'horloge et donc pourront être utilisés par la prochaine instruction.

21 : *fcl* **f** met à 0 les indicateurs basés sur f. Par exemple, un f valant 1010 mettrait les indicateurs *Zéro* et *Carry* à 0. Les indicateurs seront immédiatement accessibles au prochain coup d'horloge et donc pourront être utilisés par la prochaine instruction.

22 : *jmp* Rs**,** $Id_{\boldsymbol{hw}}$ met *PC_NEW* à $Rs + Id_{hw}$. Elle permet de faire un saut à l'adresse $Rs + Id_{hw}$. Puisque le processeur modifie *PC_NEW*, le saut ne se fera pas au prochain coup d'horloge, mais après. Donc elle ne laissera qu'une instruction s'exécuter après elle avant de sauter à l'adresse spécifiée.

23 : *jmpr* $Id_{\boldsymbol{hw}}$ est identique à *jmp*, sauf qu'elle saute à l'adresse Id_{hw} + *l'adresse actuelle*, donc $Id_{hw} + PC$.

24 : *jmpi* $Id_{\boldsymbol{hw}}$ est identique à *jmp*, sauf qu'elle saute à l'adresse Id_{hw}.

25 : *cs* Rs**,** $Id_{\boldsymbol{hw}}$ est identique à *jmp*, sauf que, juste avant de faire un saut, elle sauvegarde l'adresse actuelle dans un registre interne appelé *subroutine_return_address*, donc $PC+1$, pour après pouvoir y revenir. Ce faisant, elle saute à une "sous-routine", donc à un groupe d'instructions qui, après s'être fait exécuter, retournera à l'adresse de départ. L'analogie d'une sous-routine peut être décrite comme une "fonction" sans arguments, ou une "méthode".

26 : *csr* $Id_{\boldsymbol{hw}}$ est identique à *cs*, sauf qu'elle saute à l'adresse Id_{hw}+*l'adresse actuelle*, donc Id_{hw}+ PC.

27 : *csi* $Id_{\boldsymbol{hw}}$ est identique à *cs*, sauf qu'elle saute à l'adresse Id_{hw}.

28 : *bif* Rs**, fc,** Id_{29} est exécutée si la condition établie dans fc correspond aux indicateurs actuels. Si elle est exécutée, alors elle saute à l'adresse $Rs + Id_{29}$. Elle prend aussi longtemps pour sauter que *jmp*.

29 : *bifr* **fc,** Id_{29} est identique à *bif*, sauf qu'elle saute à l'adresse Id_{29} + *l'adresse actuelle*, donc $Id_{29} + PC$.

30 : *bifi* **fc,** Id_{29} est identique à *bif*, sauf qu'elle saute à l'adresse Id_{29}.

31 : *bio* Rs**,** $Id_{\boldsymbol{hw}}$ est exécutée si l'indicateur *Overflow* est enclenché. Si elle est exécutée, alors elle saute à l'adresse $Rs + Id_{hw}$. Elle prend aussi longtemps pour sauter que *jmp*.

32 : *bior* $Id_{\boldsymbol{hw}}$ est identique à *bio*, sauf qu'elle saute à l'adresse Id_{hw} + *l'adresse actuelle*, donc $Id_{hw} + PC$.

33 : *bioi* $Id_{\boldsymbol{hw}}$ est identique à *bio*, sauf qu'elle saute à l'adresse Id_{hw}.

34 : *ret* $Id_{\boldsymbol{hw}}$ permet de retourner à l'adresse sauvegardée dans *subroutine_return_address* par une des instructions nommées *csx*.

35 : *rti* devait être la commande de retour d'interruption, nous avons décidé de nous en passer

36 : *sas* Rd sauvegarde *subroutine_return_address* dans le registre R_d.

37 : *sal* Rs réinitialise *subroutine_return_address* à la valeur contenue dans R_s.

38 : *mov* Rd**,** Ms**,** $Id_{\boldsymbol{hw}}$**,** **s** copie 8 bytes d'information à l'adresse $Ms + Id_{hw}$ dans la mémoire qu'elle dépose dans le registre Rd. Notons que, pour copier ou non un octet, le bit respectif de cet octet-ci doit être enclenché dans s. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

39 : *mem* Md**,** Rs**,** $Id_{\boldsymbol{hw}}$**,** **s** copie 8 bytes d'information de Rs dans $Md + Id_{hw}$ dans la mémoire. s a le même effet que dans *mov*. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

40. ***memi*** Md**,** $Id_{\boldsymbol{hw}}$**, s** fixe 8 bytes dans la mémoire à Id_{hw}. Si Id_{hw} est négatif, donc si son 32ème bit est à 1, alors les bits de 33 à 64 inclus sont mis à 1, sinon 0. s a le même effet que dans *mov*. Le résultat sera immédiatement accessible au prochain coup d'horloge et donc pourra être utilisé par la prochaine instruction.

41 : ***ldi*** Rd**,** $Id_{\boldsymbol{hw}}$**, c** À l'aide de cette instruction, si c=0, alors les 32 premiers bits du registre Rd sont mis à Id_{hw}. Si c=1, alors c'est les 32 derniers bits qui sont mis à Id_{hw}. Généralement, pour mettre des valeurs par défaut dans des registres, il faut donc utiliser deux instructions pour le faire.

42 et plus : ***nop*** ne fait strictement rien.

12.6 Interview

Voici quelques questions que nous avons posées à notre expert.

Adrian : Je pense implémenter un bus de mémoire 64bit. Est-ce que c'est trop ?
Expert : Comme vous cherchez à faire un processeur très simple, et que vous voulez l'optimiser plutôt pour Tetris, vous n'avez pas besoin de beaucoup de RAM. Comme votre FPGA est petite, un bus de 64bit va prendre toute la place dans la FPGA, qui va non seulement ralentir le processeur, mais aussi ne pas laisser la place aux autres modules. Vous pensez utiliser combien de mémoire ?
Adrian : Je pense utiliser au maximum 4kb.
Expert : Donc il faudrait mieux implémenter un bus de mémoire de 16bit, ou 32bit maximum, car 16bit donne accès à 64 kilo-octets de mémoire adressable.

Adrian : Je pense faire qu'une seule interruption. Pour savoir quel périphérique a activé cette interruption, le processeur demandera à chaque appareil connecté si c'était lui.
Expert : Ceci est une bonne idée. Vous pouvez aussi faire un contrôleur d'interruptions externe au processeur, qui va multiplexer chaque interruption de chaque périphérique.

Adrian : Est-ce que c'est mieux de faire un clock, un reset et une interruption qui vont s'activer à un signal positif ou négatif ? (posedge ou negedge)
Expert : Normalement, les chips FPGA ont un Clock et un reset posedge. Ce qui compte c'est que vous restiez consistants à travers votre design, donc vous avez le choix.

Adrian : Quand j'ai une interruption, que dois-je faire avec mon pipeline ?
Expert : Quand vous avez une interruption, vous devez arrêter l'exécution de nouvelles instructions, et finir votre pipeline. Après cela, votre processeur va jump à l'adresse de l'interruption prédéfinie à l'avance.

Adrian : Mes instructions sont Big Endian, tandis que mes registres sont Little Endian. En substance, si on compte les bits de façon Little Endian (c'est-à-dire de droite à gauche), l'opcode de l'instruction commence au 63e bit et finit au 58 bit inclus (6 bit, car 64 instructions). Est-ce qu'il faudrait changer quelque chose ?
Expert : La plupart des processeurs sont Little Endian, donc il faudrait mieux que l'opcode commence au bit 0. Mais il n'y a pas beaucoup de différence de vitesse, donc comme vous l'avez fait va aussi.

Adrian : Je pense avoir une mémoire à deux ports (on peut lire à deux adresses différentes en même temps). Ce serait plus simple de lire les instructions en même temps que les données de la même mémoire, car je veux faire une architecture de Harvard (les instructions et les données sont accessibles en même temps, donc parallèlement).
Expert : Des mémoires standards n'ont qu'un seul port. De nos jours, la majorité des processeurs arrêtent leur exécution de nouvelles instructions et attendent jusqu'à ce que le processeur finisse de stocker ou de lire les données dans la mémoire. Donc il faudrait mieux que votre processeur attende aussi et halte l'exécution d'instructions pendant le transfert de données.

Adrian : Pour montrer ce que mon processeur peut faire, j'aimerais écrire Tetris comme démo dessus. Je peux programmer en assembleur, mais cela me prendra plus de temps que d'écrire Tetris en C. Quel compilateur devrais-je adapter si je devais en adapter un ?
Expert : GCC (Gnu C Compiler) est un bon compilateur C, mais il est très grand.
Adrian : Je connais aussi Clang, un compilateur LLVM, donc qui génère le même AST (Abstract Syntax Tree, un code très basique). Si j'adapte le compilateur final, donc celui qui transforme l'AST en assembleur directement, je pourrais non seulement compiler C, mais aussi C++, Rust, Swift, etc. Tout ce qui est LLVM.

Expert : Clang a un autre API que GCC, donc ce n'est pas un compilateur que tout le monde utilise et qui n'est pas standard. Ses bibliothèques standards sont autres que celles de GCC. Je pense qu'il serait meilleur d'adapter GCC que Clang pour ces raisons.

Adrian : Je pense que l'architecture de mon microprocesseur va ressembler à celle de MIPS ou à celle de ATMega. Devrais-je considérer des petits compilateurs qui supportent déjà MIPS, comme LCC (Little C compiler) ?

Expert : Oui, vous pourriez bien vous baser dessus, pour économiser du temps.

12.7 Code Source et Fichiers

Structure des fichiers :

```
/
└── src
    ├── processor
    │   ├── ALU.v
    │   ├── constants.v
    │   ├── core.v
    │   ├── display_controller.v
    │   ├── flag_mem.v
    │   ├── flag_mux.v
    │   ├── instruction_decoder.v
    │   ├── io_controller.v
    │   ├── load_store_engine.v
    │   ├── memory_controller.v
    │   ├── program_controller.v
    │   ├── reg_mem.v
    │   ├── reg_mux.v
    │   ├── tb2.v
    │   └── wrapper.v
    ├── tetris-c-algorithm
    │   └── main.cpp
    ├── compiler
    │   └── transpiler.py
    └── asm
        └── tetris.s
```

src/processor/ALU.v

```
1  /*
2  ; TOUT LE CODE CI-DESSOUS A ETE ENTIEREMENT CREE PAR Adrian Vlasov
3  ; DANS LE CADRE DE SON TRAVAIL DE MATURITE
4  ;
5  ; Gymnase de Nyon, 2024, Suisse
6  ; www.morisinc.net
7  */
8  `timescale 1ns / 1ps
9
10 module ALU(
11     clk,
12     rst,
13
14     REGX,
15     REGY,
16     ALU_REGD,
17     // ALU_CARRY_IN,
18     ALU_OP,
19     ALU_IMMEDIATE,
20
21
22     ALU_FLAG_Z,
23     ALU_FLAG_N,
24     ALU_FLAG_C,
25     ALU_FLAG_O,
26
27     FLAG_Z,
28     FLAG_N,
29     FLAG_C,
30     FLAG_O
31
32 );
33     `include "constants.v"
34
35     input clk;
```